

```

// ██████████ < START > ██████████
// FILE NAME: : types.ts
// TYPE      : .TS
// PATH      : packages/core/src/types.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < START > ██████████
// FILE-PATH: packages/core/src/types.ts
//
/**
 * @module types
 * @file FILE-PATH: packages/core/src/types.ts
 *
 * Single responsibility: contracts shared by `@pendex/compile` and
 * `@pendex/split` (and consumed by the pendex CLI shell) —the domain
 * model ({@link Config}/{@link Job}/{@link Category}/{@link Manifest})
 * and the command/view layer's shared shapes ({@link Command}/
 * {@link State}/{@link ExitState}). Neither compile nor split owns
 * these; putting them in the shell package instead would make
 * compile/split depend on the shell, inverting the real dependency
 * direction. This is the same reasoning that put `ArchiveFormat.ts`
 * here rather than in either service.
 */

import type { Theme, ThemeName } from '@pendex/theme';
export type { Theme, ThemeName };

/**
 * A single selectable entry in the interactive main menu.
 */
export interface MainMenuOptions {
  // Value returned when this option is selected.
  value: string;
  // Display label shown in the menu.
  label: string;
  // Optional secondary hint text.
  hint?: string;
}

/**
 * Defines the contract every Pendex command implements directly.
 * There is no abstract base class —composition, not inheritance.
 */
export interface Command {
  // Unique identifier for the command, used for menu selection/dispatch.
  readonly key: string;
  // Display label shown in the menu.
  readonly label: string;
  // Secondary hint text shown alongside the label.
  readonly hint: string;
  // Runs the command.
  execute(): Promise<void>;
}

/**
 * Represents the application's global state, including the
 * active theme and the loaded configuration.
 *
 * Shared dependency shape for commands that can run either inside
 * the interactive Application shell or standalone via `import.meta.main`.
 *
 * `categoryColors` is derived from the active theme's `[brand]` table
 * (see the pendex CLI's bootstrap.ts) —undefined for any theme that
 * doesn't define one, or doesn't cover a given Category. Views fall

```

```

* back to a semantic theme style (e.g. theme.bold) wherever a category
* has no brand color, so coloring by category is a pure enhancement,
* never a requirement a theme must satisfy.
*/
export interface State {
  // The process's active chainable theme.
  theme: Theme;
  // The loaded application configuration.
  config: Config;
  // Optional per-category brand colors, derived from the
  // active theme's `[brand]` table.
  categoryColors?: Partial<Record<Category, string>>;
}

export type ExitFn = (code?: number | string | null) => never;

/**
 * Defines the dependencies required by the Exit command.
 */
export interface ExitState extends State {
  // Terminates the process with an optional exit code.
  exit: ExitFn;
  // Tracks the exit code throughout execution
  exitCode: number;
}

/**
 * Defines the structure of the manifest file, which tracks
 * consolidated files and empty directories.
 *
 * `categories` maps each job's filename to the Category it was compiled
 * under —optional so a manifest written by an older Pendex version
 * still parses; anything reading it (category-colored split output)
 * degrades to no coloring when it's absent.
 */
export interface Manifest {
  // Maps each compiled output filename to the list of source
  // files consolidated into it.
  files: Record<string, string[]>;
  // Maps each compiled output filename to the {@link Category}
  // it was compiled under. Optional for backward compatibility
  // with older manifests.
  categories?: Record<string, Category>;
  // Directories that were empty at compile time and
  // so contain no consolidated files.
  emptyDirectories: string[];
}

/**
 * The set of job categories Pendex recognizes for grouping and coloring output.
 */
export type Category =
  | 'source' // Source files
  | 'web' // Web site files
  | 'style' // Web style files
  | 'terminal' // Terminal/CLI files
  | 'configuration' // Config files
  | 'documentation' // Documentation files
  | 'testing' // Test files
  | 'cspell' // CSpell config files and dictionaries
  | 'misc'; // Anything not covered by previous categories (except .svg files)

/**

```

```

* A single compile job: a named output file built from a set of include/exclude
* glob patterns.
*/
export interface Job {
    // Output filename this job produces.
    readonly filename: string;
    // Category this job's output is grouped under.
    readonly category: Category;
    // Human-readable description of what this job consolidates.
    readonly description: string;
    // Glob patterns for files to include.
    readonly include: readonly (string | undefined)[];
    // Glob patterns for files to exclude, applied after `include`.
    readonly exclude: readonly (string | undefined)[];
}

export interface ThemeConfig {
    // Name of a theme file in the themes dir (without .toml),
    // e.g. "pendex", "dracula".
    name: ThemeName;
    // Path to the themes dir.
    path: string;
}

export type CompileOutputType = 'txt' | 'pdf';

/**
 * The full application configuration, typically loaded from a config file.
 */
export interface Config {
    // Theme configuration information.
    theme: ThemeConfig;
    // Keep http(s) in files?
    http: boolean;
    // Output type
    outputType: CompileOutputType;
    // Directory compiled output files are written to.
    outputDir: string;
    // Directory split (rebuilt-from-consolidated) output is written to.
    rebuiltDir: string;
    // Glob patterns excluded from all jobs globally.
    exclude: string[];
    // The compile jobs to run.
    jobs: Job[];
}

export type StyleFunction = (text: string) => string;

export type StandaloneExecution = () => Promise<void>;

// DeepPartial helper allows us to type disk data safely
export type DeepPartial<T> = T extends object
    ? {
        [P in keyof T]?: DeepPartial<T[P]>;
    }
    : T;

/**
 * Represents the loose, unchecked data layout when loading configuration files
 * from disk before defaults are applied.
 */
export type RawDiskConfig = DeepPartial<Config> & {
    // Overriding specific properties if needed to make them completely optional

```

```

// Theme configuration information.
theme?: ThemeConfig;
// Keep http(s) in files?
http?: boolean;
// Output type
outputType?: CompileOutputType;
// Directory compiled output files are written to.
outputDir?: string;
// Directory split (rebuilt-from-consolidated) output is written to.
rebuiltDir?: string;
// Glob patterns excluded from all jobs globally.
exclude?: Array<DeepPartial<string>>;
// The compile jobs to run.
jobs?: Array<DeepPartial<Job>>;
};

// ██████████ < END > - ██████████
// FILE NAME: : types.ts
// TYPE      : .TS
// PATH      : packages/core/src/types.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < END > - ██████████

// ██████████ < START > - ██████████
// FILE NAME: : Config.ts
// TYPE      : .TS
// PATH      : packages/core/src/Config.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < START > - ██████████
// FILE-PATH: packages/core/src/Config.ts
//
/**
 * @module Config
 *
 * Single responsibility: own the process's runtime {@link Config} —
 * loading the shipped `config/config.toml` defaults, merging any
 * `runtime.config.json` override on top, and exposing both a getter
 * and mutation helpers (`reload`, `resetToDefaults`, `save`) through
 * the {@link ConfigManager} singleton.
 */

import type { BunFile } from 'bun';
import { Constants, normalizePath } from './Constants';
import type {
  Category,
  Config,
  DeepPartial,
  Job,
  RawDiskConfig,
} from './types';

/**
 * BUN PROPERTIES
 */
const BASE = {
  // The name of the current file, e.g. index.ts
  CURRENT_FILE: import.meta.file,

  // A string url to the current file, e.g.
  // file:///path/to/project/index.ts. Equivalent to import.meta.url in
  // browsers.
  CURRENT_URL: new URL(import.meta.url),

```

```

ABSOLUTE_PATH: normalizePath(Bun.fileURLToPath(new URL(import.meta.url))),

// Absolute path to the current file, e.g.
// /path/to/project/index.ts. Equivalent to __filename in CommonJS
// modules (and Node.js). An alias to import.meta.filename for Node.js
// compatibility.
CURRENT_PATH: normalizePath(import.meta.path),

// Absolute path to the directory containing the current file,
// e.g. /path/to/project. Equivalent to __dirname in CommonJS modules
// (and Node.js). An alias to import.meta.dirname for Node.js
// compatibility.
CURRENT_DIR: normalizePath(import.meta.dir),

// Indicates whether the current file is the entrypoint to the
// current bun process: true if it's executed directly by bun run,
// false if it's imported.
//ENTRY_POINT: import.meta.main,

// Resolve a module specifier (e.g. "zod" or "./file.tsx") to a
// url. Equivalent to import.meta.resolve in browsers. Example:
// import.meta.resolve("zod") returns
// "file:///path/to/project/node_modules/zod/index.ts"
//CURRENT_MODULE_URL: import.meta.resolve('./Config.ts'),

// An alias to process.env.
//ENV: import.meta.env,
CONFIG_PATH: `${normalizePath(import.meta.dir)}/../config/config.toml`,
CONFIG_KEYS: [
  'theme',
  'http',
  'outputType',
  'outputDir',
  'rebuiltDir',
  'exclude',
  'jobs',
],
THEMES_PATH: `${normalizePath(import.meta.dir)}/../../theme`,
} as const;

// Absolute path to the shipped default config TOML (`packages/core/config/`).
const BASE_CONFIG_PATH: string = BASE.CONFIG_PATH;

/**
 * Copies `source[key]` onto `target[key]` only if it's defined —
 * used to apply a partial override without clobbering base values with
 * `undefined`.
 *
 * @param target - Object to assign into (mutated in place).
 * @param source - Partial object that may or may not define `key`.
 * @param key - The key to conditionally copy.
 */
export function assignKey<T, K extends keyof T>(
  target: T,
  source: Partial<T>,
  key: K,
): void {
  const value: Partial<T>[K] = source[key];
  if (value !== undefined) {
    target[key] = value;
  }
}

```

```

/**
 * Safe Defaults
 */
const SAFE_DEFAULTS: Config = {
  theme: {
    name: 'pendex',
    path: BASE.THEMES_PATH,
  },
  http: true,
  outputType: 'txt',
  outputDir: './ALL',
  rebuiltDir: './ALL_REBUILT',
  exclude: [],
  jobs: [],
} as const;

/**
 * Singleton owner of the app's runtime Config.
 *
 * Defaults are parsed once from `config/config.toml` via `Bun.TOML.parse`.
 * If a `runtime.config.json` override exists on disk, it's merged on top
 * the first time the config is loaded. There's exactly one instance per
 * process, obtained via `ConfigManager.getInstance()`.
 *
 * `theme` is a ThemeName —the stem of a file in src/themes/ (e.g.
 * "default", "dracula") —resolved by ThemeManager, not a dark/light
 * binary. Unknown names degrade to the brand palette there, so no
 * validation is needed here.
 */
export class ConfigManager {
  private static instance: ConfigManager | null = null;

  private config: Config;

  private constructor(config: Config) {
    this.config = config;
  }

  /**
   * Reads and parses the shipped config.toml into a base Config.
   *
   * @returns The base {@link Config}, with job-level excludes
   * preserved and the runtime-config path always excluded.
   */
  private static async readTomlDefaults(): Promise<Config> {
    const tomlText: string = await Bun.file(BASE_CONFIG_PATH).text();
    const parsed = Bun.TOML.parse(tomlText) as unknown as RawDiskConfig;

    // Preserve job-specific excludes without flattening them globally
    const parsedJobs: Array<DeepPartial<Job>> =
      parsed.jobs ?? SAFE_DEFAULTS.jobs;
    const jobs: Job[] = parsedJobs.map((job: DeepPartial<Job>): Job => {
      return {
        filename: job?.filename ?? '',
        category: (job?.category as Category) ?? 'misc',
        description: job?.description ?? '',
        include: job?.include ?? [],
        exclude: job?.exclude ?? [],
      };
    });

    const result: Config = {
      theme: {

```



```

        baseJob?.category ??
        'misc',
        description:
            job?.description ?? baseJob?.description ?? '',
        include: job?.include ?? baseJob?.include ?? [],
        exclude: job?.exclude ?? baseJob?.exclude ?? [],
    };
    },
    );
}

    return merged;
} catch {
    return base;
}
}

/**
 * Lazily creates (once per process) and returns the shared instance.
 *
 * @returns The shared {@link ConfigManager} instance.
 */
public static async getInstance(): Promise<ConfigManager> {
    if (!this.instance) {
        const defaults: Config = await this.readTomlDefaults();
        const config = await this.withDiskOverrides(defaults);
        this.instance = new ConfigManager(config);
    }
    return this.instance;
}

/**
 * Resets the singleton. Primarily useful for test isolation.
 */
public static resetInstance(): void {
    this.instance = null;
}

/**
 * Current in-memory config.
 *
 * @returns The active {@link Config}.
 */
public get(): Config {
    return this.config;
}

/**
 * Re-reads config.toml + runtime.config.json and replaces the
 * cached config.
 *
 * @returns The freshly-loaded {@link Config}.
 */
public async reload(): Promise<Config> {
    const defaults = await ConfigManager.readTomlDefaults();
    this.config = await ConfigManager.withDiskOverrides(defaults);
    return this.config;
}

/**
 * Discards runtime overrides in memory (does not touch disk until
 * save() is called).
 */

```

```

    * @returns The reset, defaults-only {@link Config}.
    */
    public async resetToDefaults(): Promise<Config> {
        this.config = await ConfigManager.readTomlDefaults();
        return this.config;
    }

    /**
     * Persists the current in-memory config to runtime.config.json.
     */
    public async save(): Promise<void> {
        await Bun.write(
            Constants.RUNTIME_CONFIG_PATH,
            JSON.stringify(this.config, null, 2),
        );
    }
}

// =====< END >-=====
// FILE NAME: : Config.ts
// TYPE      : .TS
// PATH      : packages/core/src/Config.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : Service.ts
// TYPE      : .TS
// PATH      : packages/core/src/Service.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// packages/core/src/Service.ts
// FILE-PATH: packages/core/src/Service.ts

/**
 * @module Service
 *
 * Single responsibility: the shared run() lifecycle every command-level
 * service follows —prepare output, resolve items, process each one
 * with hooks, produce a summary. Compile/Split subclasses supply only
 * what's actually different between them: how to resolve items, how to
 * process one, and how to fold outcomes into a summary.
 */

/**
 * Lifecycle hooks a caller (e.g. an interactive CLI view) can observe
 * during a run. Generic over the per-item type and its outcome so
 * Compile (Job -> CompileJobResult) and Split (filename -> SplitFileOutcome)
 * both get properly-typed callbacks instead of `unknown`.
 */

export interface ServiceHooks<TItem, TOutcome> {
    // Called once, before any item is processed.
    onRunStart?: () => void | Promise<void>;
    // Called before an item starts processing.
    onItemStart?: (item: TItem) => void | Promise<void>;
    // Called after an item finishes processing successfully.
    onItemSuccess?: (item: TItem, outcome: TOutcome) => void | Promise<void>;
    // Called once, after every item has run.
    onRunSuccess?: (summary: unknown) => void | Promise<void>;
}

/**
 * Abstract base for a run() lifecycle shared by every command-level

```



```

    for (const item of items) {
      if (this.hooks.onItemStart) {
        await this.hooks.onItemStart(item);
      }

      const outcome: TOutcome = await this.processItem(item);
      outcomes.push(outcome);

      if (this.hooks.onItemSuccess) {
        await this.hooks.onItemSuccess(item, outcome);
      }
    }

    const summary: TSummary = await this.buildSummary(outcomes);

    if (this.hooks.onRunSuccess) {
      await this.hooks.onRunSuccess(summary);
    }

    return summary;
  }
}

// =====< END >-=====
// FILE NAME: : Service.ts
// TYPE      : .TS
// PATH      : packages/core/src/Service.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : bootstrap.ts
// TYPE      : .TS
// PATH      : packages/core/src/bootstrap.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/core/src/bootstrap.ts
//
/**
 * @module bootstrap
 * @file FILE-PATH: src/components/bootstrap.ts
 *
 * Single responsibility: resolve a ready-to-use `{ config, theme,
 * categoryColors }` triple, exactly once, for whichever composition root
 * needs it —the pendex CLI shell's `Application.init()`, or the
 * `import.meta.main` block in `@pendex/compile`'s `Compile.ts` /
 * `@pendex/split`'s `Split.ts` for standalone runs. Lives in
 * `@pendex/core` (not the shell package) specifically so compile/split
 * can resolve their own deps without depending on the shell —this is
 * the only file that knows Config and Theme are connected
 * (`config.theme` names the TOML file `ThemeManager` loads) and the
 * only place that translates a theme's `[brand]` table into per-Category
 * colors views can use directly.
 */

import { type PendexTheme, ThemeManager } from '@pendex/theme';
import { ConfigManager } from './Config';
import type { Category, Config, State, Theme } from './types';

/**
 * The eight recognized job categories, used to filter a theme's
 * `[brand]` table.
 */

```

```

const CATEGORIES = [
  'source',
  'web',
  'style',
  'terminal',
  'configuration',
  'documentation',
  'testing',
  'misc',
] as const satisfies readonly Category[];

/**
 * A free-form map of theme color definitions.
 *
 * This represents a raw, unvalidated string dictionary from a parsed
 * theme file, where keys represent UI components or categories (e.g.,
 * 'bookCover', 'manifest', 'source', 'terminal') and values are their
 * corresponding HEX color strings.
 */
export type BrandColors = Record<string, string>;

/**
 * Filtered theme color definitions mapped strictly to recognized Job
 * Categories.
 *
 * An object where keys are a subset of the eight literal {@link
 * Category} values and values are the HEX color strings extracted
 * from the theme's raw brand table.
 *
 * If a category is missing from this object, it means the theme did not
 * provide a specific color override for it.
 */
export type CategoryColors = Partial<Record<Category, string>>;

/**
 * The result of {@link resolveRunnerDeps}: everything a composition root
 * needs to construct views and commands.
 */
export interface ResolvedDeps {
  // The active application config
  config: Config;
  // The active chainable theme
  theme: Theme;
  // Per-category colors extracted from the theme's `[brand]` table, if any
  categoryColors?: CategoryColors;
}

export type BrandColor = BrandColors | undefined;
export type CategoryColor = CategoryColors | undefined;
export type CategoryColorFn = (brand: BrandColor) => CategoryColor;

/**
 * Picks out the Category-named entries from a theme's [brand] table.
 * A theme's [brand] table is a free-form string map (it can carry
 * anything —bookCover, manifest, scanner, ...); this only keeps the
 * entries whose key is literally one of the eight Category values, so
 * a theme opts into category coloring just by naming its brand colors
 * to match (e.g. `configuration = "#..."`, not an abbreviation of it).
 * Returns undefined (not an empty object) when the theme defines no
 * [brand] table, or none of its keys match a Category, so callers can
 * tell "no coloring available" apart from "coloring available but empty".
 *
 * @param brand - The raw `[brand]` table from a parsed theme, if any.

```

```

    @returns Per-category brand colors, or `undefined` if none apply.
    */
    export const extractCategoryColors: CategoryColorFn = (
        brand: BrandColor,
    ): CategoryColor => {
        if (!brand) return undefined;

        const colors: CategoryColors = {};
        let hasKeys = false;

        for (const category of CATEGORIES) {
            const hex: string | undefined = brand[category];
            if (hex !== undefined) {
                colors[category] = hex;
                hasKeys = true;
            }
        }

        return hasKeys ? colors : undefined;
    };

    export type ResolveDepsFn = () => Promise<ResolvedDeps>;

    /**
     * Resolves the current Config, its named Theme, and any per-Category
     * brand colors, in one call.
     *
     * @returns A {@link ResolvedDeps} triple ready to hand to a view or command.
     */
    export const resolveRunnerDeps: ResolveDepsFn =
        async (): Promise<ResolvedDeps> => {
            // Get an instance of ConfigManager, this loads application default config
            const configManager: ConfigManager = await ConfigManager.getInstance();
            // Get the default config
            const config: Config = configManager.get();

            // Get an instance of the ThemeManager and load default theme from config
            const themeManager: ThemeManager = await ThemeManager.getInstance(
                config.theme.name,
            );
            // Get the default theme
            const theme: Theme = themeManager.get();

            // Get additional 'pendex' colors from the the ThemeManager
            const pendexColors: PendexTheme = themeManager.extended();
            const brandColors: BrandColor = pendexColors.brand;

            // Extract the category colors from the 'pendex' theme
            const categoryColors: CategoryColor =
                extractCategoryColors(brandColors);

            // Return all application dependencies (i.e. state)
            return { config, theme, categoryColors } as State;
        };

    // =====< END >=====
    // FILE NAME: : bootstrap.ts
    // TYPE      : .TS
    // PATH      : packages/core/src/bootstrap.ts
    // PROCESSED: 2026-09-06T21:55:30.013Z
    // =====< END >=====

    // =====< START >=====

```



```

/**
 * Styles text using the active theme's brand color for `category`,
 * bold + that hex, when one is defined. Falls back to `fallback`
 * (typically a semantic theme style like theme.bold or theme.primary)
 * for any category the active theme's [brand] table doesn't cover —
 * including every theme that has no [brand] table at all —so a
 * view can always call this and get sensible output regardless of
 * which theme is active.
 *
 * @param category - The category to look up a brand color for.
 * @param fallback - Style function to use when no brand color is
 * defined for `category`.
 * @returns A style function: either the resolved brand-color
 * styler, or `fallback`.
 */
protected categoryStyle(
  category: Category,
  fallback: StyleFunction,
): StyleFunction {
  const hex: string | undefined = this.state.categoryColors?.[category];
  if (!hex) return fallback;
  return (text: string): string => Colors.bold(Colors.hex(hex)(text));
}
}

```

```

// ██████████< END >- ██████████
// FILE NAME: : View.ts
// TYPE      : .TS
// PATH      : packages/core/src/View.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >- ██████████

```

```

// ██████████< START >- ██████████
// FILE NAME: : ArchiveFormat.ts
// TYPE      : .TS
// PATH      : packages/core/src/ArchiveFormat.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >- ██████████
// FILE-PATH: packages/core/src/ArchiveFormat.ts
//
/**

```

```

 * @module ArchiveFormat
 *
 * Single responsibility: the on-disk shape of a Pendex archive —both
 * directions, one file.
 *
 * NO TEARLINE. Entries are delimited by their banner frames alone; the
 * old TEARLINE separator was redundant given that every entry already
 * begins and ends with a structurally recognizable 6-line frame. Parsing
 * is a small state machine: find a START frame, then scan for the END
 * frame whose PATH line matches the START frame's path, and take
 * everything between verbatim. Path-matching the END frame is what makes
 * this MORE robust than the tearline was: an archived archive's inner
 * entries carry inner paths, so their frames can never falsely terminate
 * an outer entry —full self-hosting round-trips byte-perfect. Garbage
 * between entries and unterminated frames are skipped, never fatal.
 *
 * CONTENT IS PRESERVED VERBATIM. The single '\n' on each side of the
 * content belongs to the frame; slice + join is exact, so trailing
 * newlines, empty files, leading whitespace, and CRLF endings all
 * round-trip byte-for-byte.
 */

```

```

import { baseName, extName, joinPath, normalizePath } from './Constants';

const WIDTH = 40;
const CUBE = '■';
const COMMENT = '//';

// Number of lines buildBanner() emits. The parser recognizes
// frames by this shape.
const BANNER_LINE_COUNT = 6;
// Zero-based index of the `// PATH` line inside a banner.
const PATH_LINE_INDEX = 3;

// Literal prefix of a banner's path line, used both to build
// and to recognize it.
const PATH_LINE_PREFIX = '// PATH    :';
// Tag marking the start of an archived-file frame.
const START_TAG = '-< START >';
// Tag marking the end of an archived-file frame.
const END_TAG = '-< END >';

/**
 * One file recovered from an archive: its original path and verbatim content.
 */
export interface ArchivedFile {
  // The file's original path as recorded in its banner.
  readonly originalPath: string;
  // The file's content, preserved byte-for-byte.
  readonly content: string;
}

/**
 * Builds a 6-line banner frame (start or end) for a file.
 *
 * @param filePath - Original path of the file being framed.
 * @param tag - Either {@link START_TAG} or {@link END_TAG}.
 * @returns The banner text, exactly {@link BANNER_LINE_COUNT} lines
 * joined with `\\n`.
 */
function buildBanner(
  filePath: string,
  tag: typeof START_TAG | typeof END_TAG,
): string {
  const spacer = CUBE.repeat(WIDTH);
  const extension = extName(filePath).toUpperCase().replace('.', '') || 'TXT';

  // NOTE: exactly BANNER_LINE_COUNT lines, with the PATH line at
  // PATH_LINE_INDEX —the parser's frame recognition depends on this.
  return [
    `${COMMENT}${spacer}${tag}${spacer}`,
    `${COMMENT} FILE NAME: : ${baseName(filePath)} `,
    `${COMMENT} TYPE      : ${extension}`,
    `${COMMENT} PATH       : ${filePath.replace(/\\/g, '/')}`,
    `${COMMENT} PROCESSED: ${new Date().toISOString}`,
    `${COMMENT}${spacer}${tag}${spacer}`,
  ].join('\\n');
}

/**
 * True when a full banner frame with the given tag starts at lines[index].
 *
 * @param lines - The full text, split into lines.
 * @param index - Index in `lines` where the frame is expected to start.

```

```

* @param tag - The frame tag to match ({@link START_TAG} or {@link END_TAG}).
* @returns Whether a complete, correctly-shaped frame starts at `index`.
*/
function isFrame(lines: string[], index: number, tag: string): boolean {
    return (
        index + BANNER_LINE_COUNT - 1 < lines.length &&
        (lines[index] ?? '').includes(tag) &&
        (lines[index + BANNER_LINE_COUNT - 1] ?? '').includes(tag) &&
        (lines[index + PATH_LINE_INDEX] ?? '').startsWith(PATH_LINE_PREFIX)
    );
}

/**
* Extracts the path from a `// PATH : <path>` line, or null.
*
* @param line - The candidate path line (or `undefined`).
* @returns The extracted path, or `null` if `line` is missing or blank.
*/
function extractPath(line: string | undefined): string | null {
    if (!line) return null;
    // slice past the first colon only —keeps paths containing ':' intact.
    const path = line.slice(line.indexOf(':') + 1).trim();
    return path || null;
}

/**
* Wraps one file's content in its start/end banner for archiving.
* The content is embedded verbatim —no trimming —so what goes in is
* exactly what parseArchive() gives back.
*
* @param filePath - Original path of the file being archived.
* @param content - The file's raw content, embedded verbatim.
* @returns The complete archive entry text (start banner + content +
* end banner).
*/
export function buildArchiveEntry(filePath: string, content: string): string {
    return `${buildBanner(
        filePath,
        START_TAG,
    )}\n${content}\n${buildBanner(filePath, END_TAG)}`;
}

/**
* Joins already-built archive entries into one job's .txt file body.
* A blank line between entries is purely cosmetic —the parser keys on
* banner frames, not separators.
*
* @param entries - Entries produced by {@link buildArchiveEntry}.
* @returns The concatenated archive body.
*/
export function joinArchiveEntries(entries: string[]): string {
    return entries.join('\n\n');
}

/**
* Recovers every archived file from a job's .txt body.
*
* State machine: on finding a START frame, scan forward for the END
* frame whose PATH matches the START frame's path; everything between
* is the content, verbatim. An END frame carrying a *different* path
* (e.g. an inner entry of an archived archive) is treated as content.
* Unterminated START frames and any text outside a frame pair are
* skipped —corruption is localized, never fatal, and never throws.

```

```

*
* @param rawText - The full text of a job's archive file.
* @returns Every successfully-parsed {@link ArchivedFile}, in document order.
*/
export function parseArchive(rawText: string): ArchivedFile[] {
  const lines = rawText.split('\n');
  const results: ArchivedFile[] = [];
  let i = 0;

  while (i < lines.length) {
    if (isFrame(lines, i, START_TAG)) {
      const path = extractPath(lines[i + PATH_LINE_INDEX]);
      if (path) {
        let j = i + BANNER_LINE_COUNT;
        let endIndex = -1;
        while (j <= lines.length - BANNER_LINE_COUNT) {
          if (
            isFrame(lines, j, END_TAG) &&
            extractPath(lines[j + PATH_LINE_INDEX]) === path
          ) {
            endIndex = j;
            break;
          }
          j++;
        }

        if (endIndex !== -1) {
          results.push({
            originalPath: path,
            content: lines
              .slice(i + BANNER_LINE_COUNT, endIndex)
              .join('\n'),
          });
          i = endIndex + BANNER_LINE_COUNT;
          continue;
        }
      }
      i++;
    }
  }

  return results;
}

/**
* Where one job's archive lives on disk for a given output directory.
*
* @param outputDir - Base directory compiled archives are written to.
* @param filename - The job's output filename.
* @returns The joined path to the job's archive file.
*/
export function archivePathFor(outputDir: string, filename: string): string {
  const normalizedPath = normalizePath(outputDir);
  return joinPath(normalizedPath, filename);
}

// ██████████< END >-██████████
// FILE NAME: : ArchiveFormat.ts
// TYPE      : .TS
// PATH      : packages/core/src/ArchiveFormat.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

```

```

// ██████████< START >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/core/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/core/src/index.ts

/**
 * @module @pendex/core
 *
 * Public entry point for the `@pendex/core` package: shared domain
 * types, path/constant helpers, the archive on-disk format, file
 * scanning, the base `View` class, `ConfigManager`, and the
 * `bootstrap.resolveRunnerDeps()` composition helper used by every
 * Pendex entry point (CLI shell, compile, split).
 */

export * from './types';

export {
  Constants,
  baseName,
  dirName,
  extName,
  joinPath,
  normalizePath,
  toPosixPath,
} from './Constants';

export {
  archivePathFor,
  buildArchiveEntry,
  joinArchiveEntries,
  parseArchive,
  type ArchivedFile,
} from './ArchiveFormat';

export {
  findEmptyDirectories,
  loadIgnorePatterns,
  resolveJobFiles,
} from './FileScanner';

export { View, identity } from './View';

export { ConfigManager, assignKey } from './Config';

export {
  extractCategoryColors,
  resolveRunnerDeps,
  type ResolvedDeps,
} from './bootstrap';

// ██████████< END >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/core/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : Constants.ts

```

```

// TYPE      : .TS
// PATH      : packages/core/src/Constants.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >██████████
// FILE-PATH: packages/core/src/Constants.ts
//
import path from 'node:path';

export type PathStringFn = (path: string) => string;

export type PathSegment = string | number;

export type PossiblePathSegment = PathSegment | false | null | undefined;

export type Path = (...segments: PossiblePathSegment[]) => string;

/**
 * @module Constants
 *
 * Single responsibility: generic, app-wide constants (paths, dividers)
 * and tiny path-string helpers. This file does NOT know what an
 * "archive banner" looks like —that's `core/ArchiveFormat.ts`'s job.
 * Keeping the two apart means changing the on-disk archive format never
 * requires touching this file, and vice versa.
 */

export const SEPARATOR = '/';
export const PATH_SEPARATOR_REGEX = /\//g;
export const CONSECUTIVE_SLASH_REGEX = /(?!^)(?![a-zA-Z]{2,})\\{2,}/g;

/**
 * Converts a path to POSIX-style forward slashes.
 *
 * @param filePath - Path to convert.
 * @returns `filePath` with all backslashes replaced by forward slashes.
 */
export const toPosixPath: PathStringFn = (filePath: string): string => {
  return filePath.replace(PATH_SEPARATOR_REGEX, SEPARATOR);
};

/**
 * Normalizes all slashes to a standard web-safe format.
 * Exported so other modules can validate untrusted raw config string inputs.
 *
 * @param filePath - Path to convert.
 * @returns `filePath` with all double forward slashes replaced by
 * single forward slashes.
 */
export const normalizePath: PathStringFn = (filePath: string): string => {
  const result: string = toPosixPath(filePath);
  return result.replace(CONSECUTIVE_SLASH_REGEX, SEPARATOR);
};

/**
 * Joins path segments with a single '/', collapsing accidental doubles.
 * Adjusted signature to safely accept and drop falsy conditional values.
 *
 * @param segments - Path segments to join; falsy segments are dropped.
 * @returns The joined path.
 */
export const joinPath: Path = (...segments: PossiblePathSegment[]): string => {
  const allSegments = segments.filter(Boolean) as PathSegment[];

```

```

    // Convert each segment to a posix path
    const posixPath: string = allSegments
      .map((segment: PathSegment): string => toPosixPath(String(segment)))
      .join(SEPARATOR);

    // Normalize entire combined string
    return normalizePath(posixPath);
  };

  /**
   * Basename of a path, tolerant of both '/' and '\' separators.
   *
   * @param filePath - The path to extract the basename from.
   * @returns The final path segment (filename).
   */
  export const baseName: PathStringFn = (filePath: string): string => {
    return path.basename(toPosixPath(filePath));
  };

  /**
   * Extension of a path, including the leading dot (empty string if none).
   *
   * @param filePath - The path to extract the extension from.
   * @returns The extension including its leading dot, or '' if there is none.
   */
  export const extName: PathStringFn = (filePath: string): string => {
    return path.extname(toPosixPath(filePath));
  };

  /**
   * Directory portion of a path ('.' if there is none).
   *
   * @param filePath - The path to extract the directory from.
   * @returns The directory portion of `filePath`, or '.' if there is none.
   */
  export const dirName: PathStringFn = (filePath: string): string => {
    return path.dirname(toPosixPath(filePath));
  };

  /**
   * Backing class for the {@link Constants} singleton. `BASE_DIR` and
   * anything derived from it are getters rather than frozen fields so
   * they always reflect the current `process.cwd()` —important for test
   * sandboxes that `chdir` after the module first loads.
   */
  class ConstantsManager {
    private readonly BLOCK = '█';

    private readonly WIDTH = 40;

    /**
     * The current process's working directory, read live on every access.
     */
    public readonly BASE_DIR: string = toPosixPath(process.cwd());

    /**
     * Absolute path to `.gitignore` under {@link BASE_DIR}.
     */
    public readonly GITIGNORE_PATH: string = joinPath(
      this.BASE_DIR,
      '.gitignore',
    );
  }

```

```

/**
 * Absolute path to `runtime.config.json` under {@link BASE_DIR}.
 */
public readonly RUNTIME_CONFIG_PATH: string = joinPath(
    this.BASE_DIR,
    'runtime.config.json',
);

/**
 * Default directory name for compiled output.
 */
public readonly OUTPUT_DIR = 'ALL';

/**
 * Default directory name for split (rebuilt) output.
 */
public readonly REBUILT_DIR = 'ALL_REBUILT';

/**
 * Text encoding used when reading/writing files.
 */
public readonly ENCODING = 'utf-8';

/**
 * A fixed-width horizontal rule string, used for CLI section dividers.
 */
public readonly DIVIDER = this.BLOCK.repeat(this.WIDTH);

constructor() {}
}

/**
 * Shared singleton exposing app-wide constants and live path getters.
 */
export const Constants = new ConstantsManager();

// ██████████< END >- ██████████
// FILE NAME: : Constants.ts
// TYPE      : .TS
// PATH      : packages/core/src/Constants.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >- ██████████

// ██████████< START >- ██████████
// FILE NAME: : FileScanner.ts
// TYPE      : .TS
// PATH      : packages/core/src/FileScanner.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >- ██████████
// FILE-PATH: packages/core/src/FileScanner.ts
//
/**
 * @module FileScanner
 *
 * Single responsibility: answering "which files match?" —glob
 * resolution, exclude-list loading, empty-directory detection. Nothing
 * in this file writes an archive, prints anything, or knows about
 * Config/Job shapes beyond the plain string arrays it's handed. It's
 * the "model" layer: pure enough to unit test with a temp directory
 * and zero mocks.
 */

import { dirName, toPosixPath } from './Constants';

```

```

import type { Job } from './types';

/**
 * Base glob-scan options shared by every scan in this file; `cwd` is
 * set per-call.
 */
const GLOB_OPTIONS: Bun.GlobScanOptions = {
  cwd: '',
  followSymlinks: false,
  dot: true,
  absolute: false,
};

/**
 * Reads a .gitignore-style file and converts to a flat list of patterns
 * with comments and blank lines stripped.
 *
 * @param filePath - Path to the ignore-pattern file.
 * @returns The non-comment, non-blank lines, or `[]` if the file doesn't exist.
 */
export async function loadIgnorePatterns(filePath: string): Promise<string[]> {
  const file = Bun.file(filePath);
  if (!(await file.exists())) return [];

  const content = await file.text();

  return (
    content
      .split(/\r?\n/)

      // Trim whitespace from both ends
      .map(line => line.trim())

      // Remove empty lines and comments
      .filter(
        line =>
          line.trim() &&
          !line.startsWith('#') &&
          !line.startsWith('!'),
      )
  );
}

/**
 * Resolves the file set for one job.
 *
 * A job with a non-empty `include` list matches the normal way: union of
 * its include globs, minus excludes.
 *
 * A job with an **empty** `include` list is treated as a *remainder* job
 * (e.g. `8_MISC_FILES` in config.toml, described as "all remaining
 * project files"): it scans every file under `cwd` and keeps whatever no
 * earlier job has already claimed. `claimedPaths` is shared and mutated
 * by the caller across jobs —this function only reads it, so job order
 * matters (remainder jobs must run after the jobs whose leftovers they're
 * meant to catch; config.toml lists MISC last for this reason).
 *
 * @param job - The job to resolve files for.
 * @param excludes - Glob patterns to exclude from the result.
 * @param claimedPaths - Paths already claimed by earlier jobs
 *   (read-only; not mutated here).
 * @param cwd - Directory to scan from (default `.`).
 * @returns The list of file paths this job matches.

```

```

*/
export async function resolveJobFiles(
  job: Job,
  excludes: string[],
  claimedPaths: ReadonlySet<string>,
  cwd: string = '.',
): Promise<string[]> {
  const excludeGlobs = excludes.map(pattern => new Bun.Glob(pattern));
  const matches = new Set<string>();
  const isRemainderJob = job.include.length === 0;

  // Safely combine global options with the local cwd for this specific run
  const scanOptions: Bun.GlobScanOptions = {
    ...GLOB_OPTIONS,
    cwd,
  };

  if (isRemainderJob) {
    for await (const file of new Bun.Glob('**/*').scan(scanOptions)) {
      const posixFile = toPosixPath(file);

      if (claimedPaths.has(posixFile)) continue;
      if (excludeGlobs.some(glob => glob.match(posixFile))) continue;

      matches.add(file);
    }
    return [...matches];
  }

  for (const pattern of job.include) {
    const files = new Bun.Glob(pattern!).scan(scanOptions);
    for await (const file of files) {
      const posixFile = toPosixPath(file);

      if (
        !excludeGlobs.some(exclude =>
          exclude.match(toPosixPath(posixFile)),
        )
      ) {
        matches.add(file);
      }
    }
  }
  return [...matches];
}

/**
 * Match a candidate directory against the exclude globs in BOTH
 * trailing - slash forms. Some exclude patterns are written with a
 * trailing slash("node_modules/"), some without("node_modules/**")
 * —matching only one form silently let excluded directories through
 * depending on which style the caller used.
 */
const isExcludedDir = (excludeGlobs: string[], posixDir: string): boolean => {
  return excludeGlobs.some(
    glob => glob.match(posixDir) ?? glob.match(`${posixDir}/`),
  );
};

/**
 * Finds directories under `cwd` (excluding `excludes`) that contain no
 * files or subdirectories.
 */

```

```

@param cwd - Directory to scan from
* @param excludes - Glob patterns for directories to skip.
* @returns Paths of directories with an entirely empty subtree.
*/

export async function findEmptyDirectories(
  cwd: string,
  excludes: string[],
): Promise<string[]> {
  const excludeGlobs: string[] = excludes.map(
    pattern => new Bun.Glob(pattern) as unknown as string,
  );

  const scanOptions: Bun.GlobScanOptions = {
    ...GLOB_OPTIONS,
    cwd,
  };

  const allEntries = new Set<string>();

  const dirGlobs = new Bun.Glob('**/*').scan({
    ...scanOptions,
    onlyFiles: false,
  });

  for await (const entry of dirGlobs) {
    allEntries.add(toPosixPath(entry));
  }

  const allFiles = new Set<string>();
  const nonEmptyDirs = new Set<string>();

  const fileGlobs = new Bun.Glob('**/*').scan({
    ...scanOptions,
    onlyFiles: true,
  });

  for await (const file of fileGlobs) {
    const posixFile = toPosixPath(file);
    allFiles.add(posixFile);

    let parent = dirName(posixFile);
    while (parent && parent !== '.') {
      nonEmptyDirs.add(parent);
      parent = dirName(parent);
    }
  }

  const allDirs = new Set<string>();
  for (const entry of allEntries) {
    if (allFiles.has(entry)) continue;
    if (entry && !isExcludedDir(excludeGlobs, entry)) allDirs.add(entry);
  }

  const emptyDirs: string[] = [];
  for (const dir of allDirs) {
    if (!nonEmptyDirs.has(dir)) {
      emptyDirs.push(dir);
    }
  }

  return emptyDirs;
}

// ██████████ < END > ██████████
// FILE NAME: : FileScanner.ts

```

```

// TYPE      : .TS
// PATH      : packages/core/src/FileScanner.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────██████████

// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
// FILE NAME: : ThemeManager.ts
// TYPE      : .TS
// PATH      : packages/theme/src/ThemeManager.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
// FILE-PATH: packages/theme/src/ThemeManager.ts
//
//
/**
 * @module ThemeManager
 *
 * Single responsibility: own the process's active Theme, exactly the way
 * `ConfigManager` owns the active Config. Loads `src/themes/<name>.toml`
 * via `Bun.TOML.parse`, hands the raw object to
 * `ThemePalette.parsePendexTheme()` for validation, builds a
 * `DefaultTheme` from its `colors` through `ThemePalette.buildTheme`,
 * wraps it in `useTheme`'s chaining engine, and hands the same `Theme`
 * instance to every caller. Unknown theme names, missing files, or
 * malformed TOML degrade to the brand palette, never crash —the theme
 * is cosmetic and must never take the app down.
 */

import type { BunFile } from 'bun';
import type { DefaultTheme, PendexTheme, ThemeName } from './ThemePalette';
import { buildTheme, parsePendexTheme } from './ThemePalette';
import type { Theme } from './useTheme';
import { useTheme } from './useTheme';

/**
 * Directory containing `<name>.toml` theme files (`packages/theme/themes/`).
 */
const THEMES_DIR = `${import.meta.dir}/../themes`;

export interface ThemeManagerState {
  themeName: ThemeName;
  defaultTheme: DefaultTheme;
  theme: Theme;
  pendexTheme: PendexTheme;
}

/**
 * Process-wide singleton that owns the active {@link Theme}. Use
 * {@link ThemeManager.getInstance} to obtain it; the constructor is
 * private.
 */
export class ThemeManager {
  private static instance: ThemeManager | null = null;
  private static instancePromise: Promise<ThemeManager> | null = null;

  private readonly theme: Theme;
  private readonly themeName: ThemeName;
  private readonly pendexTheme: PendexTheme;
  private readonly defaultTheme: DefaultTheme;

  private constructor(state: ThemeManagerState) {
    this.themeName = state.themeName;
    this.theme = state.theme;
    this.pendexTheme = state.pendexTheme;
  }

```

```

    this.defaultTheme = state.defaultTheme;
}

/**
 * Builds the absolute path for a theme file stem under `THEMES_DIR`.
 */
private static filePathFor(name: ThemeName): string {
    return `${THEMES_DIR}/${name}.toml`;
}

/**
 * Reads and validates `src/themes/<name>.toml` into a full PendexTheme.
 *
 * @param name - Theme name (file stem under `THEMES_DIR`).
 * @returns The validated {@link PendexTheme}, degrading to the
 * brand palette on any failure.
 */
private static async readPendexTheme(
    name: ThemeName,
): Promise<PendexTheme> {
    const filePath: string = this.filePathFor(name);
    const file: BunFile = Bun.file(filePath);

    try {
        if (!(await file.exists()))
            return parsePendexTheme(undefined, name);

        const parsed: unknown = Bun.TOML.parse(await file.text());
        return parsePendexTheme(parsed, name);
    } catch {
        return parsePendexTheme(undefined, name);
    }
}

/**
 * Resolves all runtime theme state from a requested theme name:
 * - the parsed/validated full PendexTheme
 * - the flat DefaultTheme
 * - the chainable Theme proxy
 *
 * Keeping this assembly in one place avoids duplicating theme-build
 * knowledge across the singleton lifecycle.
 */
private static async loadState(
    themeName: ThemeName,
): Promise<ThemeManagerState> {
    const pendexTheme: PendexTheme = await this.readPendexTheme(themeName);
    const defaultTheme: DefaultTheme = buildTheme(pendexTheme.colors);
    const theme: Theme = useTheme(defaultTheme);

    return {
        themeName,
        defaultTheme,
        theme,
        pendexTheme,
    };
}

/**
 * Lazily creates (once per process) and returns the shared instance.
 * The first call decides the theme for the process; subsequent calls
 * return the same instance regardless of the name passed —same
 * contract as ConfigManager.
 */

```

```

*
* Uses an in-flight promise guard so concurrent first calls do not
* race and build multiple instances.
*
* @param themeName - Theme to load on first call (default
*   `'pendex'`); ignored on subsequent calls.
* @returns The shared {@link ThemeManager} instance.
*/
public static async getInstance(
  themeName: ThemeName = 'pendex',
): Promise<ThemeManager> {
  if (this.instance) return this.instance;
  if (this.instancePromise) return this.instancePromise;

  this.instancePromise = (async () => {
    const state: ThemeManagerState = await this.loadState(themeName);
    const instance = new ThemeManager(state);
    this.instance = instance;
    this.instancePromise = null;
    return instance;
  })();

  return this.instancePromise;
}

/**
* Resets the singleton. Primarily useful for test isolation.
*/
public static resetInstance(): void {
  this.instance = null;
  this.instancePromise = null;
}

/**
* The active chainable Theme, built from the ten ThemeColors slots.
*/
public get(): Theme {
  return this.theme;
}

/**
* The name the active theme was loaded as.
*
* Note: this is the requested file stem, not necessarily the theme
* metadata's internal `name` field. If the requested file is missing
* or malformed, the returned Theme may still be a fallback-built one.
*/
public name(): ThemeName {
  return this.themeName;
}

/**
* The full parsed theme file, including extended tables (palette,
* ansi, syntax, git, diagnostics, website, terminal, diff, brand)
* that buildTheme() doesn't consume. Undefined for any table not
* present in the loaded file.
*/
public extended(): PendexTheme {
  return this.pendexTheme;
}

/**
* The active flat DefaultTheme before it is wrapped in `useTheme`.

```

```

    * Useful for tests or for call sites that want direct style
    * functions without the chainable proxy.
    */
    public raw(): DefaultTheme {
        return this.defaultTheme;
    }
}

// ██████████< END >-██████████
// FILE NAME: : ThemeManager.ts
// TYPE      : .TS
// PATH      : packages/theme/src/ThemeManager.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : ThemePalette.ts
// TYPE      : .TS
// PATH      : packages/theme/src/ThemePalette.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/theme/src/ThemePalette.ts
//
/**
 * @module ThemePalette
 *
 * Single responsibility: what a theme *means* —the {@link DefaultTheme}
 * contract, the hex-map schema that TOML theme files must satisfy, and
 * the builder ({@link buildTheme}) that turns a hex map into a working
 * `DefaultTheme` via `Colors`. No picocolors, no file I/O (`ThemeManager`
 * owns loading), no chaining (`useTheme` owns that).
 *
 * {@link BRAND_PALETTE} below is the in-code copy of `src/themes/pendex.toml`
 * —pendex.toml is the canonical theme every other theme file's layout
 * mirrors, and its colors are the ones verified (WCAG-AA 4.5:1 for
 * text, 3:1 for the intentionally de-emphasized `muted`) against both
 * of Pendex's standard terminal backgrounds, #1A1A1A and #262626. This
 * constant exists so the app always has a synchronous fallback theme
 * (commands' `deps.theme ?? FallbackTheme` can't await a file read) and
 * so a missing/corrupt themes directory degrades gracefully instead of
 * crashing —the old light cream/brown palette (the original asset-pack
 * "Default" theme, background #F6F1E8) has been retired: every Pendex
 * surface, including the synchronous fallback, is now dark-terminal
 * first.
 */

import { Colors } from '@pendex/color';

/**
 * A theme is chosen by name —the stem of a file in src/themes/.
 */
export type ThemeName = string;

/**
 * The required root color slots every theme file provides.
 *
 * Keeping this list as a single constant prevents drift between:
 * - the ThemeColors type,
 * - TOML parsing/validation,
 * - and the buildTheme() implementation.
 */
export const THEME_COLOR_KEYS = [
    'primary',

```

```

    'secondary',
    'success',
    'warning',
    'error',
    'info',
    'muted',
    'foreground',
    'background',
    'titleBg',
] as const;

export type ThemeColorKey = (typeof THEME_COLOR_KEYS)[number];

/**
 * The hex slots every theme file provides. Structural styles (bold,
 * italic, ...) are behavior, not palette, and stay in code rather than
 * in this interface.
 */
export type ThemeColors = Readonly<Record<ThemeColorKey, string>>;

/**
 * The fully-resolved, ready-to-use theme: every slot is a function that
 * takes plain text and returns ANSI-styled text. This is what
 * `useTheme()` consumes to build its chainable proxy API.
 */
export interface DefaultTheme {
    /** @category Semantic Utilities */
    primary(text: string): string;
    /** @category Semantic Utilities */
    secondary(text: string): string;
    /** @category Semantic Utilities */
    success(text: string): string;
    /** @category Semantic Utilities */
    warning(text: string): string;
    /** @category Semantic Utilities */
    error(text: string): string;
    /** @category Semantic Utilities */
    info(text: string): string;
    /** @category Semantic Utilities */
    muted(text: string): string;

    /** @category Typography */
    title(text: string): string;
    /** @category Typography */
    subtitle(text: string): string;

    /** @category CSS Styles */
    color(text: string): string;
    /** @category CSS Styles */
    backgroundColor(text: string): string;
    /** @category CSS Styles */
    bold(text: string): string;
    /** @category CSS Styles */
    italic(text: string): string;
    /** @category CSS Styles */
    textDecoration(text: string): string;
    /** @category CSS Styles */
    textTransform(text: string): string;
}

/**
 * The key form of {@link DefaultTheme}, used anywhere a single theme slot
 * name needs to be carried around (e.g. `useTheme`'s pending style stack).

```

```

*/
export type DefaultThemeKey = keyof DefaultTheme;

/**
 * Pendex brand palette (mirrors src/themes/pendex.toml's root colors).
 * Every value here clears WCAG-AA (4.5:1) against both #1A1A1A and
 * #262626, except `muted` (3:1 —intentionally de-emphasized text).
 */
export const BRAND_PALETTE: ThemeColors = {
  primary: '#D6A448', // Brass
  secondary: '#6F92B0', // Blueprint
  success: '#799470', // Olive
  warning: '#D99C5A', // Copper
  error: '#C57967', // Brick
  info: '#6AAF85', // Teal
  muted: '#7B746B', // Ash
  foreground: '#E9E0D2', // Parchment
  background: '#1A1A1A', // Charcoal
  titleBg: '#262626', // Graphite
};

/**
 * Builds a working {@link DefaultTheme} from a hex map. This is the
 * ONLY place that decides how palette slots combine with structural
 * styles.
 *
 * @param colors - The ten resolved hex color slots for the theme.
 * @returns A fully-usable {@link DefaultTheme} with every slot styled
 * via `Colors`.
 */
export function buildTheme(colors: ThemeColors): DefaultTheme {
  const semantic = {
    primary: Colors.hex(colors.primary),
    secondary: Colors.hex(colors.secondary),
    success: Colors.hex(colors.success),
    warning: Colors.hex(colors.warning),
    error: Colors.hex(colors.error),
    info: Colors.hex(colors.info),
    muted: Colors.hex(colors.muted),
  };

  return {
    ...semantic,

    title: txt =>
      Colors.bgHex(colors.primary)(
        Colors.bold(Colors.hex(colors.titleBg)` ${txt} `)),
    subtitle: txt => Colors.dim(Colors.hex(colors.foreground)(txt)),

    color: Colors.hex(colors.foreground),
    backgroundColor: txt => Colors.bgHex(colors.background)` ${txt} `,
    bold: Colors.bold,
    italic: Colors.italic,
    textDecoration: Colors.underline,
    textTransform: txt => txt.toUpperCase(),
  };
}

/**
 * Synchronous fallback theme (brand palette) for contexts that can't
 * await ThemeManager.
 */

```

```

export const FALLBACK_THEME: DefaultTheme = buildTheme(BRAND_PALETTE);

/**
 * @PendexTheme —the full pendex.toml schema
 *
 * A theme file is more than the ten ThemeColors slots buildTheme() needs:
 * it can also carry a named swatch palette, ANSI 16-color mappings,
 * syntax-highlighting colors, git/diagnostic/diff colors, website colors,
 * and Pendex's own brand-concept colors (compile/split/manifest/...).
 * None of those extra tables are consumed by buildTheme() today —they're
 * captured and exposed via ThemeManager.extended() for future use (e.g.
 * coloring a job's title by its Category using [brand]) without forcing
 * every theme author to define slots nothing reads yet.
 *
 * *FORMAT NOTE:
 * color scalars (primary, secondary, ..., titleBg) sit at the ROOT of a
 * theme TOML file, not nested under a [colors] table —only the extended
 * groups below get their own [section]. That's what pendex.toml (and
 * default/dracula/tokyonight/onedark/monokaipro files) actually look like;
 * extractColors() reads accordingly.
 */

/**
 * The full schema a `src/themes/*.toml` file may satisfy: the ten
 * required {@link ThemeColors} slots plus optional metadata and
 * extended color tables. Extended tables aren't consumed by
 * {@link buildTheme} —they're captured for future use (e.g. via
 * `ThemeManager.extended()`) without forcing every theme author to
 * define slots nothing reads yet.
 */
export interface PendexTheme {
  name: string;
  author?: string;
  version?: string;
  description?: string;
  colors: ThemeColors;
  palette?: Record<string, string>;
  ansi?: Record<string, string>;
  syntax?: Record<string, string>;
  git?: Record<string, string>;
  diagnostics?: Record<string, string>;
  website?: Record<string, string>;
  terminal?: Record<string, string>;
  diff?: Record<string, string>;
  brand?: Record<string, string>;
}

type ExtendedTableKey =
  | 'palette'
  | 'ansi'
  | 'syntax'
  | 'git'
  | 'diagnostics'
  | 'website'
  | 'terminal'
  | 'diff'
  | 'brand';

const EXTENDED_TABLE_KEYS: ExtendedTableKey[] = [
  'palette',
  'ansi',
  'syntax',
  'git',

```

```

    'diagnostics',
    'website',
    'terminal',
    'diff',
    'brand',
];

type UnknownRecord = Record<string, unknown>;

interface MutableTheme extends PendexTheme {
    palette?: Record<string, string>;
    ansi?: Record<string, string>;
    syntax?: Record<string, string>;
    git?: Record<string, string>;
    diagnostics?: Record<string, string>;
    website?: Record<string, string>;
    terminal?: Record<string, string>;
    diff?: Record<string, string>;
    brand?: Record<string, string>;
}

/**
 * Returns true only for plain object-ish records we can safely index.
 */
function isRecord(value: unknown): value is UnknownRecord {
    return typeof value === 'object' && value !== null;
}

/**
 * Returns true only for strings that look like #RRGGBB hex colors.
 */
function isHexColor(value: unknown): value is string {
    return typeof value === 'string' && /^[0-9A-Fa-f]{6}$/.test(value.trim());
}

/**
 * Normalizes a hex color scalar to a stable uppercase #RRGGBB form.
 */
function normalizeHex(value: string): string {
    return value.trim().toUpperCase();
}

/**
 * Extracts the required root ThemeColors from a parsed TOML object,
 * falling back slot-by-slot to {@link BRAND_PALETTE}.
 */
function extractColors(obj: UnknownRecord): ThemeColors {
    const entries = THEME_COLOR_KEYS.map(key => {
        const raw = obj[key];
        const value = isHexColor(raw) ? normalizeHex(raw) : BRAND_PALETTE[key];
        return [key, value] as const;
    });

    return Object.fromEntries(entries) as ThemeColors;
}

/**
 * Extracts an optional extended table, preserving only string:string
 * entries. Non-object or empty tables are treated as absent.
 */
function extractTable(
    obj: UnknownRecord,
    key: ExtendedTableKey,

```

```

): Record<string, string> | undefined {
    const raw = obj[key];
    if (!isRecord(raw)) return undefined;

    const entries = Object.entries(raw).filter(([, value]) => {
        return typeof value === 'string';
    });

    if (entries.length === 0) return undefined;

    return Object.fromEntries(entries) as Record<string, string>;
}

/**
 * Validates/normalizes a parsed theme TOML object into a full
 * {@link PendexTheme}, degrading safely to the brand palette anywhere
 * fields are missing or malformed.
 *
 * @param raw - Parsed TOML object, if any.
 * @param fallbackName - Theme file stem requested by the caller; used
 * when the theme metadata itself does not provide a name.
 * @returns A fully-formed {@link PendexTheme}.
 */
export function parsePendexTheme(
    raw: unknown,
    fallbackName: ThemeName = 'pendex',
): PendexTheme {
    const obj: UnknownRecord = isRecord(raw) ? raw : {};

    // Initialize and cast as a Partial version of the mutable type
    const theme = {
        name: typeof obj.name === 'string' ? obj.name : fallbackName,
        author: typeof obj.author === 'string' ? obj.author : undefined,
        version: typeof obj.version === 'string' ? obj.version : undefined,
        description:
            typeof obj.description === 'string' ? obj.description : undefined,
        colors: extractColors(obj),
    } as Partial<MutableTheme> as MutableTheme; // Double assertion tells TS it will be filled

    // The loop safely populates the remaining keys
    for (const key of EXTENDED_TABLE_KEYS) {
        theme[key] = extractTable(obj, key);
    }

    return theme;
}

// =====< END >-=====
// FILE NAME: : ThemePalette.ts
// TYPE      : .TS
// PATH      : packages/theme/src/ThemePalette.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/theme/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/theme/src/index.ts

/**

```

```

* @module @pendex/theme
*
* Public entry point for the `@pendex/theme` package: the singleton
* {@link ThemeManager}, the palette/schema types from `ThemePalette`,
* the {@link useTheme} chaining engine, and the synchronous
* {@link FallbackTheme} for contexts that can't await a file read.
*/

import { FALLBACK_THEME } from './ThemePalette';
import { useTheme } from './useTheme';

export { ThemeManager } from './ThemeManager';

export {
  BRAND_PALETTE,
  buildTheme,
  FALLBACK_THEME,
  parsePendexTheme,
  THEME_COLOR_KEYS,
  type DefaultTheme,
  type DefaultThemeKey,
  type PendexTheme,
  type ThemeColorKey,
  type ThemeColors,
  type ThemeName,
} from './ThemePalette';

export { useTheme, type ChainableTheme, type Theme } from './useTheme';

/**
 * Synchronous brand-palette theme.
 *
 * This is the chainable proxy form of {@link FALLBACK_THEME}; exporting
 * both lets callers choose either the flat style-function object or the
 * chainable `theme.bold.primary('txt')` API.
 */
export const FallbackTheme = useTheme(FALLBACK_THEME);

// ██████████< END >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/theme/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : useTheme.ts
// TYPE      : .TS
// PATH      : packages/theme/src/useTheme.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/theme/src/useTheme.ts
//
/**
 * @module useTheme
 *
 * Single responsibility: the chaining/composition *mechanism* —turning a
 * flat {@link DefaultTheme} object into a chainable, callable API via a
 * `Proxy`. This file has no idea what colors mean (that's
 * `ThemePalette.ts`); it only knows how to stack style functions and
 * resolve them to a string. It's the piece that already does what the
 * rest of this refactor is chasing —favoring composition over
 * inheritance —so it's left structurally as-is, just relocated out of

```

```

* the file that used to also hold the palette data.
*/

import type { DefaultTheme, DefaultThemeKey } from './ThemePalette';

/**
 * Hybrid structure matching:
 * - a string (via `toString`/`valueOf`/`Symbol.toPrimitive`)
 * - a direct text constructor function (`theme('text')`)
 * - recursive chain attributes (`theme.bold.primary('text')`)
 *
 * @example
 * ```ts
 * const theme: Theme = useTheme(defaultTheme);
 * String(theme.bold.primary('Hello')); // bold + primary-colored "Hello"
 * ```
 */
export interface ChainableTheme {
  (txt?: string): ChainableTheme & string;

  // Semantic Utilities
  /** @category Semantic Utilities */
  primary: ChainableTheme & string;
  secondary: ChainableTheme & string;
  success: ChainableTheme & string;
  warning: ChainableTheme & string;
  error: ChainableTheme & string;
  info: ChainableTheme & string;
  muted: ChainableTheme & string;
  foreground: ChainableTheme & string;
  background: ChainableTheme & string;
  titleBg: ChainableTheme & string;

  // Typography Styles
  /** @category Typography Styles */
  pill: ChainableTheme & string;
  /** @category Typography Styles */
  title: ChainableTheme & string;
  /** @category Typography Styles */
  subtitle: ChainableTheme & string;

  // Additional Styling Functionality
  /** @category Additional Styling Functionality */
  color: ChainableTheme & string;
  /** @category Additional Styling Functionality */
  backgroundColor: ChainableTheme & string;
  /** @category Additional Styling Functionality */
  bold: ChainableTheme & string;
  /** @category Additional Styling Functionality */
  italic: ChainableTheme & string;
  /** @category Additional Styling Functionality */
  textDecoration: ChainableTheme & string;
  /** @category Additional Styling Functionality */
  textTransform: ChainableTheme & string;

  /** Resolves the accumulated chain to a plain styled string. */
  toString(): string;
}

/**
 * A {@link ChainableTheme} that also behaves as a plain string.
 */
export type Theme = ChainableTheme & string;

```

```

/**
 * Builds a chainable, composable {@link Theme} from a flat
 * {@link DefaultTheme}. Each property access returns a new proxied
 * `Theme` node so styles can be stacked (`theme.bold.primary`) before
 * being applied to text, either by calling the chain (`theme('text')`)
 * or by coercing it to a string.
 *
 * @param activeTheme - The flat, already-resolved theme whose styler
 * functions back this chain.
 * @param initialTxt - Text already accumulated in the chain (internal
 * recursion state).
 * @param pendingStyles - Style keys queued to apply once text is
 * supplied (internal recursion state).
 * @returns A {@link Theme} proxy supporting both function-call and
 * chained-property styling.
 */
export const useTheme = (
  activeTheme: DefaultTheme,
  initialTxt: string = '',
  pendingStyles: DefaultThemeKey[] = [],
): Theme => {
  // Computes styles on the text whenever string
  // conversion is triggered or new text is supplied
  const compile = (textToStyle: string): string => {
    return pendingStyles.reduce(
      (result, styleKey) => activeTheme[styleKey](result),
      textToStyle,
    );
  };

  // The handler function allows calling the chain
  // instance like a function: theme('TEXT') or
  // theme.bold('TEXT')
  const targetFunction = (txt?: string) => {
    const textToProcess = txt ?? initialTxt ?? '';
    const compiledText = compile(textToProcess);

    // Return a fresh theme node so operations can
    // keep chaining off the outcome
    return useTheme(activeTheme, compiledText, []);
  };

  return new Proxy(targetFunction, {
    get(_, prop) {
      // These symbols/methods intercept conversions
      // to primitives (strings) automatically
      if (
        prop === 'toString' ||
        prop === 'valueOf' ||
        prop === Symbol.toPrimitive
      ) {
        return () => compile(initialTxt);
      }

      if (prop in activeTheme) {
        const styleKey = prop as keyof DefaultTheme;

        // If text is already present, apply style eagerly;
        // otherwise, stack it up
        if (initialTxt) {
          const nextText = activeTheme[styleKey](initialTxt);
          return useTheme(activeTheme, nextText, pendingStyles);
        }
      }
    },
  });
}

```

```

    }

    return useTheme(activeTheme, initialTxt, [
      ...pendingStyles,
      styleKey,
    ]);
  }
  return undefined;
},
}) as unknown as Theme;
};

// =====< END >-=====
// FILE NAME: : useTheme.ts
// TYPE      : .TS
// PATH      : packages/theme/src/useTheme.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : CompileService.ts
// TYPE      : .TS
// PATH      : packages/compile/src/CompileService.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/compile/src/CompileService.ts

/**
 * @module CompileService
 *
 * Single responsibility: *do* the compile —
 *   resolve files,
 *   write archives,
 *   build the manifest
 *   hand back plain data describing what happened.
 *
 * All state changes live here, nothing renders here.
 *
 * Imports `FileScanner` (what matches) and `ArchiveFormat` (how an entry is
 * written).
 *
 * Never imports `@clack/prompts` —that import boundary is what
 * makes this file testable without mocking a terminal, and reusable if
 * Pendex ever grows a non-CLI front end.
 */

import type { Config, Job, Manifest } from '@pendex/core';
import {
  archivePathFor,
  buildArchiveEntry,
  Constants,
  findEmptyDirectories,
  joinArchiveEntries,
  joinPath,
  loadIgnorePatterns,
  resolveJobFiles,
} from '@pendex/core';
import type { BunFile } from 'bun';
import { mkdir, rm } from 'node:fs/promises';
import { ToPdf, ToPdfService } from '../../utils/ToPdf';

/**
 * Accumulators shared across every job in a compile run, before

```

```

    * any specific job is attached.
    */
export interface CompileJobContext {
    // Application configuration.
    readonly config: Config;
    // Combined global excludes (config + .gitignore).
    readonly excludes: string[];
    // The manifest being built up across all jobs.
    readonly manifest: Manifest;
    // Paths already claimed by earlier jobs.
    readonly claimedPaths: Set<string>;
    // Compile hooks.
    readonly hooks: CompileHooks;
    // Current working directory.
    readonly cwd: string;
}

/**
 * Everything {@link compileJob} needs to resolve, archive, and write
 * one job's output.
 */
export interface CompileJob extends CompileJobContext {
    // The job to compile.
    readonly job: Job;
    // Directory the job's archive file is written into.
    readonly outputDir: string;
}

/**
 * Outcome of compiling a single job.
 */
export interface CompileJobResult {
    // The job that was compiled.
    readonly job: Job;
    // Number of files this job matched and archived.
    readonly fileCount: number;
}

/**
 * Full summary returned after a compile run completes.
 */
export interface CompileSummary {
    // Combined global excludes used for this run.
    readonly excludes: string[];
    // Per-job outcomes, in the order jobs ran.
    readonly jobOutcomes: CompileJobResult[];
    // Total files archived across every job.
    totalFiles: number;
    // Directories found with an entirely empty subtree.
    readonly emptyDirectories: string[];
}

/**
 * Optional progress callbacks a caller (e.g. the interactive CLI
 * view) can hook into a compile run.
 */
export interface CompileHooks {
    // Called once, before any job runs, with the resolved global excludes.
    onCompileStart?: (excludes: string[]) => void | Promise<void>;

    // Called before a job starts compiling.
    onJobStart?: (job: Job) => void | Promise<void>;
}

```

```

    // Called after a job finishes compiling successfully.
    onJobSuccess?: (
      job: Job,
      outcome: CompileJobResult,
    ) => void | Promise<void>;

    // Called once, after every job has run, with the total file count.
    onCompileSuccess?: (totalFiles: number) => void | Promise<void>;
  }

/**
 * Resolves all active exclude glob patterns by merging global config defaults
 * with project-level `.gitignore` rules.
 *
 * @param config Active Pendex configuration object.
 * @param cwd Current working directory context. Defaults to process.cwd().
 * @returns A promise resolving to an array of consolidated exclude
 * glob strings.
 */
export const resolveExcludes = async (config: Config): Promise<string[]> => {
  const excludes: string[] = [...config.exclude];
  const gitignoreFile: BunFile = Bun.file(Constants.GITIGNORE_PATH);

  // Bun's blazing fast file checking
  if (await gitignoreFile.exists()) {
    const gitignorePatterns: string[] = await loadIgnorePatterns(
      Constants.GITIGNORE_PATH,
    );
    excludes.push(...gitignorePatterns);
  }

  return excludes;
};

/**
 * Wipes and recreates the output directory.
 *
 * @param dir - Directory to wipe and recreate.
 */
export const prepareOutputDirectory = async (dir: string): Promise<void> => {
  await rm(dir, { recursive: true, force: true });
  await mkdir(dir, { recursive: true });
};

/**
 * Resolves, archives, and writes one job's .txt file.
 * Mutates `manifest` and `claimedPaths` as a side effect.
 *
 * @param currentJob - The job plus the shared per-run accumulators.
 * @returns The job and how many files it archived.
 */
export const compileJob = async (
  currentJob: CompileJob,
): Promise<CompileJobResult> => {
  const {
    job,
    excludes,
    outputDir,
    manifest,
    claimedPaths,
    config,
  }: CompileJob = currentJob;

```

```

const combinedExcludes = [
  ...new Set([...excludes, ...job.exclude]),
] as string[];
const files: string[] = await resolveJobFiles(
  job,
  combinedExcludes,
  claimedPaths,
);

if (files.length === 0) return { job, fileCount: 0 };

manifest.files[job.filename] = files.map(f => f.replace(/\\/g, '/'));
manifest.categories ??= {};
manifest.categories[job.filename] = job.category;
files.forEach(f => claimedPaths.add(f.replace(/\\/g, '/')));

const entries = await Promise.all(
  files.map(async filePath => {
    const content = await Bun.file(filePath).text();
    return buildArchiveEntry(filePath, content);
  }),
);

const txtPath = archivePathFor(outputDir, job.filename);
const archiveContent = joinArchiveEntries(entries);

// If PDF output type is specified in config, convert .txt to .pdf
if (config?.outputType === 'pdf') {
  const tempTxtPath = `${txtPath}.tmp`;
  const pdfFilename: string = job.filename.replace(/\.txt$/i, '.pdf');
  const pdfPath: string = archivePathFor(outputDir, pdfFilename);

  // Write temp text archive
  await Bun.write(tempTxtPath, archiveContent);

  // Convert text archive to PDF using ToPdf utility
  const converter: ToPdfService = ToPdf.create({
    fontSize: 7,
    lineHeight: 11,
    oneLongPage: false,
    syntaxHighlighting: true,
  });
  await converter.convertToPdf(tempTxtPath, pdfPath);

  // Remove temporary text file via Bun Shell
  await Bun.$`rm -f ${tempTxtPath}`;

  // Update manifest mapping to reflect PDF archive filename
  manifest.files[pdfFilename] = manifest.files[job.filename] as string[];
  delete manifest.files[job.filename];
  manifest.categories[pdfFilename] = job.category;
  delete manifest.categories[job.filename];
} else {
  // Default plain-text output
  await Bun.write(txtPath, archiveContent);
}

return {
  job,
  fileCount: files.length,
};
};

```

```

/**
 * Writes `manifest.json` into the output directory.
 *
 * @param outputDir - Directory to write `manifest.json` into.
 * @param manifest - The manifest data to serialize.
 */
export const writeManifest = async (
  outputDir: string,
  manifest: Manifest,
): Promise<void> => {
  await Bun.write(
    joinPath(outputDir, 'manifest.json'),
    JSON.stringify(manifest, null, 2),
  );
};

/**
 * Headless execution path used by standard commands and isolated testing tools.
 *
 * @param config - The active application config.
 * @param hooks - Optional progress callbacks.
 * @returns The full {@link CompileSummary} for this run.
 */
export const runCompile = async (
  config: Config,
  hooks?: CompileHooks,
): Promise<CompileSummary> => {
  await prepareOutputDirectory(config.outputDir);

  const cwd: string = Constants.BASE_DIR;
  const excludes: string[] = await resolveExcludes(config);
  const emptyDirectories: string[] = await findEmptyDirectories(
    cwd,
    excludes,
  );
  const manifest: Manifest = {
    files: {},
    emptyDirectories,
  };
  const claimedPaths: Set<string> = new Set<string>();

  const result: CompileSummary = {
    excludes,
    jobOutcomes: [],
    totalFiles: 0,
    emptyDirectories,
  };

  if (hooks?.onCompileStart) {
    await hooks.onCompileStart(excludes);
  }

  for (const job of config.jobs as Job[]) {
    if (hooks?.onJobStart) {
      await hooks.onJobStart(job);
    }

    const currentCompileJob: CompileJob = {
      config,
      job,
      excludes,
      outputDir: config.outputDir,
      manifest,
    };
  }

```

```

        claimedPaths,
        hooks: hooks ?? {},
        cwd,
    };

    const outcome: CompileJobResult = await compileJob(currentCompileJob);

    result.jobOutcomes.push(outcome);
    result.totalFiles += outcome.fileCount;

    if (hooks?.onJobSuccess) {
        await hooks.onJobSuccess(job, outcome);
    }
}

await writeManifest(config.outputDir, manifest);

if (hooks?.onCompileSuccess) {
    await hooks.onCompileSuccess(result.totalFiles);
}

return result;
};

/**
 * Initializes a new compilation session, resolving global excludes
 * and preparing the manifest and claimedPaths set.
 *
 * @param config Application configuration object.
 * @param hooks Optional progress event hook callbacks.
 * @param cwd Current working directory context. Defaults to Constants.BASE_DIR.
 * @returns A promise resolving to the initialized compilation session state.
 */
export const initializeCompile = async (
    config: Config,
    hooks?: CompileHooks,
    cwd: string = Constants.BASE_DIR,
): Promise<CompileJobContext> => {
    const excludes: string[] = await resolveExcludes(config);
    const claimedPaths: Set<string> = new Set<string>();

    const manifest: Manifest = {
        files: {},
        categories: {},
        emptyDirectories: await findEmptyDirectories(cwd, excludes),
    };

    return {
        config,
        excludes,
        manifest,
        claimedPaths,
        hooks: hooks ?? {},
        cwd,
    };
};

/**
 * Compiles a single job against the shared accumulators from {@link
 * initializeCompile}.
 *
 * @param job - The job to compile.
 * @param ctx - Shared concrete config and per-run accumulators.

```

```

    * @returns The job's compile outcome.
    */
export const compileSingleJob = async (
  job: Job,
  ctx: CompileJobContext,
): Promise<CompileJobResult> => {
  return await compileJob({
    ...ctx,
    job,
    excludes: ctx.excludes,
    outputDir: ctx.config.outputDir,
    manifest: ctx.manifest,
    claimedPaths: ctx.claimedPaths,
    config: ctx.config,
  });
};

/**
 * Finishes a manually-sequenced compile run: finds empty directories
 * and writes the manifest.
 *
 * @param config - The active application config.
 * @param ctx - Shared concrete excludes and manifest accumulated
 * during the run.
 * @returns A {@link CompileSummary} template for the caller to augment.
 */
export const finalizeCompile = async (
  config: Config,
  ctx: CompileJobContext,
): Promise<CompileSummary> => {
  ctx.manifest.emptyDirectories = await findEmptyDirectories(
    ctx.cwd,
    ctx.excludes,
  );
  await writeManifest(config.outputDir, ctx.manifest);

  return {
    excludes: ctx.excludes,
    jobOutcomes: [],
    totalFiles: 0,
    emptyDirectories: ctx.manifest.emptyDirectories,
  };
};

// =====< END >-=====
// FILE NAME: : CompileService.ts
// TYPE      : .TS
// PATH      : packages/compile/src/CompileService.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : Compile.bin.ts
// TYPE      : .TS
// PATH      : packages/compile/src/Compile.bin.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/compile/src/Compile.bin.ts
//
import type { ResolvedDeps, StandaloneExecution } from '@pendex/core';
import { Exit } from '@pendex/exit';
import { Compile } from './Compile';

```

```

/**
 * STANDALONE EXECUTION ENTRY POINT
 * `bun run src/commands/Compile.ts` —only pulls a Config, no
 * App/State required.
 */

const standaloneExecution: StandaloneExecution = async (): Promise<void> => {
  console.log();

  const { resolveRunnerDeps } = await import('@pendex/core');
  const state: ResolvedDeps = await resolveRunnerDeps();

  const compile = new Compile(state);
  await compile.execute();

  const exit = new Exit({
    ...state,
    exit: process.exit.bind(process),
    exitCode: 0,
  });
  await exit.execute();
};

if (import.meta.main) {
  await standaloneExecution();
}

// =====< END >=====
// FILE NAME: : Compile.bin.ts
// TYPE      : .TS
// PATH      : packages/compile/src/Compile.bin.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >=====

// =====< START >=====
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/compile/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >=====
// FILE-PATH: packages/compile/src/index.ts

/**
 * @module @pendex/compile
 *
 * Public entry point for the `@pendex/compile` package:
 *
 *   `Compile` command
 *   `CompileView` renderer
 *   `CompileService` functions (`runCompile`, `compileJob`, etc.)
 */

export { Compile } from './Compile';

export {
  compileJob,
  compileSingleJob,
  finalizeCompile,
  initializeCompile,
  prepareOutputDirectory,
  resolveExcludes,
  runCompile,
  writeManifest,
  type CompileHooks,

```

```

    type CompileJob,
    type CompileJobContext,
    type CompileJobResult,
    type CompileSummary,
  } from './CompileService';

export { CompileView } from './CompileView';

// =====< END >-=====
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/compile/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : Compile.ts
// TYPE      : .TS
// PATH      : packages/compile/src/Compile.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/compile/src/Compile.ts

/**
 * @module Compile
 *
 * Single responsibility: satisfy `Command` for "compile" —hold its
 * menu identity (key/label/hint) and hand its deps to `CompileView`. No
 * rendering happens here: the outro that used to be banded into this
 * file now lives where the rest of the clack session lives, in
 * `CompileView` (a view owns intro THROUGH outro). Construction only
 * needs a `Config`, which keeps it standalone-runnable.
 */

import type { Command, State } from '@pendex/core';
import { CompileView } from './CompileView';

/**
 * The "Compile Codebase" command:
 * consolidates project files per the active config's jobs.
 */
export class Compile implements Command {
  private readonly state: State;
  private readonly view: CompileView;

  readonly key: string;
  readonly label: string;
  readonly hint: string;

  private readonly STRINGS = {
    key: 'compile',
    label: 'Compile project',
    hint: 'Consolidates project files',
  } as const;

  constructor(state: State) {
    this.state = state;
    this.view = new CompileView(this.state);

    this.key = this.STRINGS.key;
    this.label = `${this.state.theme.secondary(this.STRINGS.label)}`;
    this.hint = `${this.state.theme.secondary(this.STRINGS.hint)}`;
  }
}

```

```

/**
 * Runs the compile command by delegating to its {@link CompileView}.
 */
async execute(): Promise<void> {
    await this.view.render();
}
}

// =====< END >-=====
// FILE NAME: : Compile.ts
// TYPE      : .TS
// PATH      : packages/compile/src/Compile.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : CompileView.ts
// TYPE      : .TS
// PATH      : packages/compile/src/CompileView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/compile/src/CompileView.ts

/**
 * @module CompileView
 *
 * Single responsibility: render the compile flow to the terminal —the
 * only `@clack/prompts` caller for `compile`.
 *
 * A VIEW OWNS ITS WHOLE CLACK SESSION, intro THROUGH outro. Every exit
 * path here (success, error) closes the session with `outro()`. This is
 * what fixes the dangling-session look when Compile runs standalone —
 * previously the outro lived nowhere (bandaid: in the Command, which
 * put presentation in the wrong layer). Inside the interactive shell
 * this is also correct: each command render is its own complete clack
 * session, and `Application` clears the console between sessions anyway.
 */

import type { NoteOptions, Task } from '@clack/prompts';
import { intro, log, note, outro, tasks } from '@clack/prompts';
import { Colors } from '@pendex/color';
import type { Config, Job, State, StyleFunction, Theme } from '@pendex/core';
import { View } from '@pendex/core';
import type { CompileJobContext, CompileJobResult } from './CompileService';
import {
    compileSingleJob,
    finalizeCompile,
    initializeCompile,
} from './CompileService';

export interface ResultSummary {
    outputDir: string;
    fileCount: string;
    jobCount: string;
}

/**
 * Renders the interactive compile session: intro, per-job progress,
 * summary, and outro.
 */
export class CompileView extends View {
    // User-facing strings for this view.

```

```

private readonly STRINGS = {
  title: ' @ RUNNING PENDEX COMPILATION ',
  excludedTitle: 'Excluded Patterns',
  presentAction: 'Compiling',
  pastAction: 'compiled',

  resultTitle: 'Compilation Results',
  resultDir: 'Files created in',
  resultTotal: 'Total files compiled',
  resultJobs: 'Total jobs compiled',

  error: 'Failed to compile project files.',
  outroSuccess: 'Compilation complete',
  outroFailure: 'Compilation failed —see errors above',
} as const;

private totalFiles: number = 0;

constructor(state: State) {
  super(state);
}

/**
 * Runs and renders the full compile session, from intro to outro.
 */
public override render = async (): Promise<void> => {
  const theme: Theme = this.state.theme;
  const config: Config = this.state.config;

  try {
    // Initialize compilation assets
    const ctx: CompileJobContext = await initializeCompile(config);

    intro(`${theme.title(this.STRINGS.title)}`);

    this.renderExcludes(ctx.excludes.join(', '));

    // Build tasks that execute live service methods inside Clack's runner
    const jobTasks: Task[] = await this.buildJobTasks(ctx);

    // Clack runs tasks one-by-one, showing live spinner progress
    await tasks(jobTasks);

    // Write manifest and clean up folders
    await finalizeCompile(config, ctx);

    // Render final summary block
    this.renderSummary(this.totalFiles);

    const success: string = `${this.STRINGS.outroSuccess.toUpperCase()}`;
    log.success(Colors.bgGreen(Colors.black(success)));
  } catch (err: unknown) {
    log.error(`${theme.error(this.STRINGS.error)}`);
    if (err instanceof Error) log.error(`${theme.error(err.message)}`);
    outro(Colors.bgRed(Colors.black(this.STRINGS.outroFailure)));
  }
};

private buildJobTasks = async (ctx: CompileJobContext): Promise<Task[]> => {
  const theme: Theme = this.state.theme;
  const config: Config = this.state.config;

  return config.jobs.map((job: Job): Task => {

```

```

const rawDesc: string = job.description || job.filename;
const titleStyle: StyleFunction = this.categoryStyle(
    job.category,
    theme.bold,
);
const resultStyle: StyleFunction = this.categoryStyle(
    job.category,
    theme.primary,
);

return {
    title: `${this.STRINGs.presentAction} ${titleStyle(rawDesc)}`,
    task: async (): Promise<string> => {
        const outcome: CompileJobResult = await compileSingleJob(
            job,
            {
                ...config,
                ...ctx,
            },
        );

        const warningCount: string = `${theme.warning(`${outcome.fileCount}`)}`;
        const styledCount: string = `${theme.bold(warningCount)}`;

        const rawDesc1: string = `${rawDesc.charAt(0).toUpperCase()}`;
        const rawDesc2: string = `${rawDesc.slice(1).toLowerCase()}`;
        const combinedDesc: string = `${rawDesc1}${rawDesc2} ${this.STRINGs.pastAction}`;
        const formattedDesc: string = `${resultStyle(combinedDesc)}`;

        this.totalFiles += outcome.fileCount;
        return `${styledCount} ${formattedDesc}`;
    },
};
});
});

private renderExcludes = (excludes: string): void => {
    const message: string = excludes;
    const title: string = this.STRINGs.excludedTitle;
    const options: NoteOptions = {
        format: this.mutedFormatter,
    };

    note(message, title, options);
};

/**
 * Renders the post-compile summary note (output dir, file count,
 * job count).
 *
 * @param fileCount - Total files compiled across all jobs.
 */
private renderSummary = (fileCount: number): void => {
    const config: Config = this.state.config;

    const rawResult: ResultSummary = {
        outputDir: this.directoryFormatter(config.outputDir),
        fileCount: this.numberFormatter(fileCount),
        jobCount: this.numberFormatter(config.jobs.length),
    };

    const result: ResultSummary = {
        outputDir: `${this.STRINGs.resultDir}: ${rawResult.outputDir}`,
    };

```

```

        fileCount: `${this.STRINGs.resultTotal}: ${rawResult.fileCount}`,
        jobCount: `${this.STRINGs.resultJobs}: ${rawResult.jobCount}`,
    };

    const message: string = `${result.outputDir}\n${result.fileCount}\n${result.jobCount}`;
    const title: string = `${this.STRINGs.resultTitle}`;
    const options: NoteOptions = {
        format: this.successFormatter,
    };

    note(message, title, options);
};

private mutedFormatter = (text: string): string =>
    `${this.state.theme.muted(text)}`;

private successFormatter = (text: string): string =>
    `${this.state.theme.success(text)}`;

private directoryFormatter = (text: string): string =>
    `${this.state.theme.warning(`"/${text}"`)}`;

private numberFormatter = (number: number): string =>
    `${this.state.theme.bold.primary(String(number))}`;
}

// ██████████< END >- ██████████
// FILE NAME: : CompileView.ts
// TYPE      : .TS
// PATH      : packages/compile/src/CompileView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >- ██████████

// ██████████< START >- ██████████
// FILE NAME: : Exit.ts
// TYPE      : .TS
// PATH      : packages/exit/src/Exit.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >- ██████████
// FILE-PATH: packages/exit/src/Exit.ts
//
/**
 * @module Exit
 *
 * Single responsibility: say goodbye and terminate the process. Still
 * deliberately NOT split into command + service + view —one `outro()`
 * and a `process.exit()` don't earn three files. Note: this outro closes
 * the SHELL's clack session (the menu `Application` opened); the
 * per-command sessions close themselves inside their views.
 */

import type { Command, ExitState } from '@pendex/core';
import { ExitView } from './ExitView';

/**
 * The "Exit Program" command: prints a goodbye message and terminates
 * the process.
 */
export class Exit implements Command {
    private readonly state: ExitState;
    private readonly view: ExitView;

    public readonly key: string;

```

```

public readonly label: string;
public readonly hint: string;

private readonly STRINGS = {
    key: 'exit',
    label: 'Exit program',
    hint: 'Terminate system process',
} as const;

constructor(state: ExitState) {
    this.state = state;
    this.view = new ExitView(this.state);

    this.key = this.STRINGS.key;
    this.label = `${this.state.theme.secondary(this.STRINGS.label)}`;
    this.hint = `${this.state.theme.secondary(this.STRINGS.hint)}`;
}

/**
 * Prints the goodbye message and terminates the process via
 * `state.exit` (or `process.exit` as a fallback).
 */
public async execute(): Promise<void> {
    await this.view.render();

    const code: number = this.state.exitCode;

    if (this.state.exit) {
        this.state.exit(code);
    } else {
        process.exit(code);
    }
}
}

// =====< END >-=====
// FILE NAME: : Exit.ts
// TYPE      : .TS
// PATH      : packages/exit/src/Exit.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : ExitView.ts
// TYPE      : .TS
// PATH      : packages/exit/src/ExitView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
// FILE-PATH: packages/exit/src/ExitView.ts
//
import { outro } from '@clack/prompts';
import { View, type State } from '@pendex/core';

export class ExitView extends View {
    private readonly STRINGS = {
        goodbye: '👋 Exiting @pendex. System process ended.',
    } as const;

    constructor(state: State) {
        super(state);
    }

    public override async render(): Promise<void> {

```

```

    // Format exit message
    const goodbye: string = this.state.theme.primary.muted(
        this.STRINGS.goodbye,
    );

    outro(goodbye);
}
}

// ██████████< END >-██████████
// FILE NAME: : ExitView.ts
// TYPE      : .TS
// PATH      : packages/exit/src/ExitView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/exit/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/exit/src/index.ts

/**
 * @module @pendex/exit
 *
 * Public entry point for the `@pendex/exit` package:
 *
 * `Exit` command
 */

export { Exit } from './Exit';

// ██████████< END >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/exit/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : Colors.ts
// TYPE      : .TS
// PATH      : packages/color/src/Colors.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/color/src/Colors.ts
/**
 * @module Colors
 *
 * Single responsibility: turning text into ANSI-styled text.
 *
 * Drop-in replacement for picocolors —same call shape
 * - `Colors.red('x')`
 * - `Colors.bold('x')`
 * - `Colors.bgMagenta('x')`
 *
 * plus what picocolors can't do and the TOML theme system needs:
 * - 24-bit truecolor via {@link Colors.hex}
 * - 24-bit truecolor via {@link Colors.bgHex}
 */

```

```

import type { Env } from 'bun';

/**
 * Types
 */
export type Styler = (text: string) => string;
export type StyleFactory = (open: string, close: string) => Styler;
export type CodeStyleFactory = (code: number) => Styler;
export type BooleanFn = () => boolean;
export type Tuple = [number, number, number];
export type HexStringParser = (hex: string) => Tuple;

/**
 * Constants
 */
export const ESC: string = '\x1b[';

/**
 * Determines whether ANSI color output should be enabled, honoring the
 * `NO_COLOR`/`FORCE_COLOR` conventions and falling back to a TTY check.
 * Resolved once at module load and cached in {@link enabled}.
 *
 * @returns `true` if color output should be enabled, `false` otherwise.
 */
export const detectColorSupport: BooleanFn = (): boolean => {
  const env: Env & NodeJS.ProcessEnv & ImportMetaEnv = Bun.env;

  if ('NO_COLOR' in env) return false;

  if (env.FORCE_COLOR !== undefined && env.FORCE_COLOR !== '0') {
    return true;
  }

  return process.stdout?.isTTY === true;
};

/**
 * Module-level flag controlling whether stylers emit ANSI codes.
 */
let enabled: boolean = detectColorSupport();

/**
 * Builds a styler for an open/close code pair. Like picocolors, any
 * occurrence of the close code inside the text is replaced with the
 * open code so nested styles of the same family survive.
 *
 * @param open - ANSI escape sequence that opens the style.
 * @param close - ANSI escape sequence that closes the style.
 * @returns A {@link Styler} that wraps input text in the given codes.
 */
export const style: StyleFactory = (open: string, close: string): Styler => {
  return (text: string): string => {
    if (!enabled) return text;

    // Handle cases where runtime data does not match TypeScript types
    if (typeof text !== 'string') {
      return `${open}${String(text ?? '').replaceAll(
        close,
        open,
      )}${close}`;
    }

    return `${open}${text.replaceAll(close, open)}${close}`;
  };
};

```

```

    };
};

/**
 * Builds a standard-palette foreground {@link Styler} for a given SGR code.
 *
 * @param code - ANSI SGR foreground color code (e.g. 31 for red).
 * @returns A {@link Styler} for that foreground color.
 */
export const fg: CodeStyleFactory = (code: number): Styler =>
    style(`${ESC}${code}m`, `${ESC}39m`);

/**
 * Builds a standard-palette background {@link Styler} for a given SGR code.
 *
 * @param code - ANSI SGR background color code (e.g. 41 for red).
 * @returns A {@link Styler} for that background color.
 */
export const bg: CodeStyleFactory = (code: number): Styler =>
    style(`${ESC}${code}m`, `${ESC}49m`);

/**
 * Parses a hex color string into its RGB components.
 *
 * @param hex - A `'#rgb'` or `'#rrggbb'` string (leading `#` optional).
 * @returns A `[r, g, b]` tuple with each channel in the range 0-255.
 * @throws {Error} If `hex` is not a valid 3- or 6-digit hex color.
 */
export const parseHex: HexStringParser = (hex: string): Tuple => {
    let value: string = hex.startsWith('#') ? hex.slice(1) : hex;

    if (value.length === 3) {
        value = value
            .split('')
            .map((ch: string): string => ch + ch)
            .join('');
    }

    if (!/^([0-9a-fA-F]{6})$/.test(value)) {
        throw new Error(`Colors: invalid hex color "${hex}"`);
    }

    const num: number = parseInt(value, 16);

    return [(num >> 16) & 0xff, (num >> 8) & 0xff, num & 0xff];
};

/**
 * Static namespace of ANSI text stylers. A drop-in replacement for
 * picocolors, plus 24-bit truecolor support via {@link Colors.hex} and
 * {@link Colors.bgHex}. Every property returns a {@link Styler} —a
 * function that wraps a string in the corresponding ANSI codes, or
 * returns the string unchanged when color output is disabled (see
 * {@link Colors.isEnabled}).
 *
 * @example
 * ```ts
 * console.log(Colors.bold(Colors.red('error:')), 'something broke');
 * console.log(Colors.hex('#ff8800')('custom orange'));
 * ```
 */
export class Colors {
    private constructor() {}

```

```

// Whether color output is currently enabled.
static get isEnabled(): boolean {
    return enabled;
}

// Force-enables ANSI color output, overriding auto-detection.
static enable(): void {
    enabled = true;
}

// Force-disables ANSI color output, overriding auto-detection.
static disable(): void {
    enabled = false;
}

// @category Modifiers
static readonly reset: Styler = style(`${ESC}0m`, `${ESC}0m`);
static readonly bold: Styler = style(`${ESC}1m`, `${ESC}22m`);
static readonly dim: Styler = style(`${ESC}2m`, `${ESC}22m`);
static readonly italic: Styler = style(`${ESC}3m`, `${ESC}23m`);
static readonly underline: Styler = style(`${ESC}4m`, `${ESC}24m`);
static readonly inverse: Styler = style(`${ESC}7m`, `${ESC}27m`);
static readonly strikethrough: Styler = style(`${ESC}9m`, `${ESC}29m`);

// @category Standard foreground
static readonly black: Styler = fg(30);
static readonly red: Styler = fg(31);
static readonly green: Styler = fg(32);
static readonly yellow: Styler = fg(33);
static readonly blue: Styler = fg(34);
static readonly magenta: Styler = fg(35);
static readonly cyan: Styler = fg(36);
static readonly white: Styler = fg(37);

// @category Bright foreground
static readonly gray: Styler = fg(90);
static readonly blackBright: Styler = fg(90);
static readonly redBright: Styler = fg(91);
static readonly greenBright: Styler = fg(92);
static readonly yellowBright: Styler = fg(93);
static readonly blueBright: Styler = fg(94);
static readonly magentaBright: Styler = fg(95);
static readonly cyanBright: Styler = fg(96);
static readonly whiteBright: Styler = fg(97);

// @category Standard background
static readonly bgBlack: Styler = bg(40);
static readonly bgRed: Styler = bg(41);
static readonly bgGreen: Styler = bg(42);
static readonly bgYellow: Styler = bg(43);
static readonly bgBlue: Styler = bg(44);
static readonly bgMagenta: Styler = bg(45);
static readonly bgCyan: Styler = bg(46);
static readonly bgWhite: Styler = bg(47);

// @category Bright background
static readonly bgBlackBright: Styler = bg(100);
static readonly bgRedBright: Styler = bg(101);
static readonly bgGreenBright: Styler = bg(102);
static readonly bgYellowBright: Styler = bg(103);
static readonly bgBlueBright: Styler = bg(104);
static readonly bgMagentaBright: Styler = bg(105);

```

```

static readonly bgCyanBright: Styler = bg(106);
static readonly bgWhiteBright: Styler = bg(107);

/**
 * 24-bit truecolor foreground from a hex string —what TOML
 * themes are built on.
 */
static hex(hexColor: string): Styler {
  const [r, g, b]: Tuple = parseHex(hexColor);
  return style(`${ESC}38;2;${r};${g};${b}m`, `${ESC}39m`);
}

/**
 * 24-bit truecolor background from a hex string.
 */
static bgHex(hexColor: string): Styler {
  const [r, g, b]: Tuple = parseHex(hexColor);
  return style(`${ESC}48;2;${r};${g};${b}m`, `${ESC}49m`);
}
}

```

```

// ██████████< END >- ██████████
// FILE NAME: : Colors.ts
// TYPE      : .TS
// PATH      : packages/color/src/Colors.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >- ██████████

```

```

// ██████████< START >- ██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/color/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >- ██████████
// FILE-PATH: packages/color/src/index.ts

```

```

/**
 * @module @pendex/color
 *
 * Public entry point for the `@pendex/color` package. Re-exports
 * {@link Colors}, the ANSI text-styling namespace, and the
 * {@link Styler} function type it is built from.
 */

```

```

export {
  Colors,
  ESC,
  bg,
  detectColorSupport,
  fg,
  parseHex,
  style,
  type BooleanFn,
  type CodeStyleFactory,
  type HexStringParser,
  type StyleFactory,
  type Styler,
  type Tuple,
} from './Colors';

```

```

// ██████████< END >- ██████████
// FILE NAME: : index.ts
// TYPE      : .TS

```

```
// PATH      : packages/color/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < END > - ██████████

// ██████████ < START > - ██████████
// FILE NAME: : Split.ts
// TYPE      : .TS
// PATH      : packages/split/src/Split.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < START > - ██████████
// FILE-PATH: packages/split/src/Split.ts

/**
 * @module Split
 *
 * Single responsibility: satisfy `Command` for "split" —hold its menu
 * identity and hand its deps to `SplitView`. See `Compile.ts` for the
 * fuller explanation of this file's (deliberately thin) role.
 */

import type { Command, State } from '@pendex/core';
import { SplitView } from './SplitView';

/**
 * The "Split Archive" command: rebuilds original files from compiled archives.
 */

export class Split implements Command {
  private readonly state: State;
  private readonly view: SplitView;

  readonly key: string;
  readonly label: string;
  readonly hint: string;

  private readonly STRINGS = {
    key: 'split',
    label: 'Split Archive',
    hint: 'Rebuild project files',
  } as const;

  constructor(state: State) {
    this.state = state;
    this.view = new SplitView(this.state);

    this.key = this.STRINGS.key;
    this.label = `${this.state.theme.secondary(this.STRINGS.label)}`;
    this.hint = `${this.state.theme.secondary(this.STRINGS.hint)}`;
  }

  /**
   * Runs the split command by delegating to its {@link SplitView}.
   */
  async execute(): Promise<void> {
    await this.view.render();
  }
}

// ██████████ < END > - ██████████
// FILE NAME: : Split.ts
// TYPE      : .TS
// PATH      : packages/split/src/Split.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < END > - ██████████
```

```

// ██████████< START >────────────────██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/split/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────██████████
// FILE-PATH: packages/split/src/index.ts

/**
 * @module @pendex/split
 *
 * Public entry point for the `@pendex/split` package: the `Split`
 * command, its `SplitView` renderer, and the headless `SplitService`
 * functions (`runSplit`, `splitArchiveFile`, etc.) that back both the
 * interactive and standalone/test call paths.
 */

export { Split } from './Split';

export {
  initializeSplit,
  prepareRebuildDirectory,
  readManifest,
  restoreEmptyDirectories,
  runSplit,
  splitArchiveFile,
  splitSingleFile,
  type SplitFileOutcome,
  type SplitHooks,
  type SplitSummary,
} from './SplitService';

export { SplitView } from './SplitView';

// ██████████< END >────────────────██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/split/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >────────────────██████████

// ██████████< START >────────────────██████████
// FILE NAME: : SplitView.ts
// TYPE      : .TS
// PATH      : packages/split/src/SplitView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────██████████
// FILE-PATH: packages/split/src/SplitView.ts

/**
 * @module SplitView
 * @file FILE-PATH: src/views/SplitView.ts
 *
 * Single responsibility: render the split flow to the terminal —the
 * only `@clack/prompts` caller for `split`. Owns its whole clack
 * session: every exit path (success, missing manifest, error) closes
 * with `outro()`, so a standalone run never leaves the session
 * dangling.
 */

import { intro, log, note, outro, tasks, type Task } from '@clack/prompts';
import { Colors } from '@pendex/color';

```

```

import type { Config, Manifest, State, Theme } from '@pendex/core';
import { View } from '@pendex/core';
import { initializeSplit, splitSingleFile } from './SplitService';

/**
 * Renders the interactive split session: intro, per-file progress,
 * summary, and outro.
 */
export class SplitView extends View {
  // User-facing strings for this view.
  private readonly STRINGS = {
    title: ' ⚙️ RUNNING PENDEX SPLIT ',
    excludedTitle: 'Excluded Patterns',
    presentAction: 'Splitting from',
    noManifest: 'No manifest found in',
    pleaseCompile: 'Please compile first.',
    success: 'Entire directory state restored inside',
    error: 'Splitting sequence crashed.',
    missingSource: 'Source .txt file not found, skipping.',
    filesRecreated: 'files recreated from this archive.',
    totallabel: 'Total files recreated',
    outroSuccess: 'Split complete',
    outroNoManifest: 'Nothing to split.',
    outroFailure: 'Split failed —see errors above',
  } as const;

  constructor(state: State) {
    super(state);
  }

  /**
   * Runs and renders the full split session, from intro to outro.
   */
  public override async render(): Promise<void> {
    const theme: Theme = this.state.theme;
    const config: Config = this.state.config;

    const outputDir: string = config.outputDir;
    const rebuiltDir: string = config.rebuiltDir;
    const exclude: string[] = config.exclude;

    intro(theme.title(this.STRINGS.title));

    try {
      const manifest: Manifest | null = await initializeSplit(
        outputDir,
        rebuiltDir,
      );

      if (!manifest) {
        log.error(
          `${theme.error(`${this.STRINGS.noManifest} /`
            + `${outputDir}. ${this.STRINGS.pleaseCompile}`)}\n`,
        );
        outro(theme.muted(this.STRINGS.outroNoManifest));
        return;
      }

      if (exclude.length > 0) {
        note(exclude.join(', '), this.STRINGS.excludedTitle, {
          format: this.mutedFormatter,
        });
      }
    }
  }
}

```

```

    const outcomes: number[] = [];

    const fileTasks: Task[] = Object.keys(manifest.files).map(
      (filename, index) =>
        this.buildFileTask(filename, index, manifest, outcomes),
    );

    await tasks(fileTasks);

    const totalRecreated = outcomes.reduce(
      (sum, count) => sum + (count || 0),
      0,
    );

    const styledRebuiltDir = theme.muted('/' + rebuiltDir);
    log.step(
      `${theme.success(`${this.STRINGS.success}
${styledRebuiltDir}.`)}`,
    );

    const styledTotal = theme.bold(
      theme.warning(totalRecreated.toString()),
    );
    log.success(
      `${theme.success(this.STRINGS.totalLabel)}: ${styledTotal}`,
    );

    const success = this.STRINGS.outroSuccess.toUpperCase();
    log.success(Colors.bgGreen(Colors.black(success)));
  } catch (err) {
    log.error(theme.error(this.STRINGS.error));
    if (err instanceof Error) {
      log.error(theme.error(err.message));
    }
    outro(theme.error(this.STRINGS.outroFailure));
  }
}

/**
 * Builds a single localized execution block formatted for the
 * Clack task pipeline.
 */
private buildFileTask(
  filename: string,
  index: number,
  manifest: Manifest,
  outcomes: number[],
): Task {
  const theme: Theme = this.state.theme;
  const config: Config = this.state.config;

  const category = manifest.categories?.[filename];
  const titleStyle = category
    ? this.categoryStyle(category, theme.bold)
    : theme.bold;

  return {
    title: `${this.STRINGS.presentAction} ${titleStyle(filename)}`,
    task: async (): Promise<string> => {
      const outcome = await splitSingleFile(
        config.outputDir,
        config.rebuiltDir,

```

```

        filename,
    );

    if (!outcome.archiveFound) {
        outcomes[index] = 0;
        return theme.muted(this.STRINGs.missingSource);
    }

    outcomes[index] = outcome.filesRecreated;
    const styledCount = theme.bold.warning(
        outcome.filesRecreated.toString(),
    );
    return `${styledCount}
${theme.primary(this.STRINGs.filesRecreated)}';
    },
};
}

```

```

private mutedFormatter = (text: string) =>
    `${this.state.theme.muted(text)}';
}

```

```

// ██████████< END >-- ██████████
// FILE NAME: : SplitView.ts
// TYPE      : .TS
// PATH      : packages/split/src/SplitView.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-- ██████████

```

```

// ██████████< START >-- ██████████
// FILE NAME: : SplitService.ts
// TYPE      : .TS
// PATH      : packages/split/src/SplitService.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-- ██████████
// FILE-PATH: packages/split/src/SplitService.ts

```

```

/**
 * @module SplitService
 *
 * Single responsibility: *do* the split —read the manifest, parse each
 * archive, rewrite the original files —and hand back plain data. Same
 * "hook" role as `CompileService`: depends on `ArchiveFormat` for
 * parsing, never imports `@clack/prompts`.
 */

```

```

import { mkdir, rm } from 'node:fs/promises';

```

```

import {
    dirName,
    joinPath,
    normalizePath,
    parseArchive,
    type Manifest,
} from '@pendex/core';

```

```

/**
 * Outcome of splitting a single job's archive back into real files.
 */

```

```

export interface SplitFileOutcome {
    // The job's archive filename.
    readonly filename: string;
    // Whether the archive file existed on disk.

```

```

    readonly archiveFound: boolean;
    // Number of files successfully recreated from this archive.
    readonly filesRecreated: number;
}

/**
 * Full summary returned after a split run completes.
 */
export interface SplitSummary {
    // Whether `manifest.json` was found in the output directory.
    readonly manifestFound: boolean;
    // Per-file outcomes, in manifest order.
    readonly fileOutcomes: SplitFileOutcome[];
    // Total files recreated across every archive.
    readonly totalFilesRecreated: number;
}

/**
 * Optional progress callbacks a caller (e.g. the interactive CLI
 * view) can hook into a split run.
 */
export interface SplitHooks {
    // Called once, after the manifest is read, before any files are split.
    onSplitStart?: (manifest: Manifest) => void | Promise<void>;
    // Called before a job's archive starts splitting.
    onFileStart?: (filename: string) => void | Promise<void>;
    // Called after a job's archive finishes splitting.
    onFileSuccess?: (
        filename: string,
        outcome: SplitFileOutcome,
    ) => void | Promise<void>;
}

/**
 * Reads and parses manifest.json from an output directory, or null if
 * it doesn't exist.
 *
 * @param outputDir - Directory to read `manifest.json` from.
 * @returns The parsed {@link Manifest}, or `null` if the file doesn't exist.
 */
export async function readManifest(
    outputDir: string,
): Promise<Manifest | null> {
    const normalizedPath = normalizePath(outputDir);
    const manifestFile = Bun.file(joinPath(normalizedPath, 'manifest.json'));

    if (!(await manifestFile.exists())) {
        return null;
    }

    return manifestFile.json() as Promise<Manifest>;
}

/**
 * Wipes the rebuild directory natively.
 *
 * @param dir - Directory to remove.
 */
export async function prepareRebuildDirectory(dir: string): Promise<void> {
    const normalizedPath = normalizePath(dir);

    await rm(normalizedPath, {
        recursive: true,
    });
}

```

```

        force: true,
    });
}

/**
 * Recreates every empty directory recorded in the manifest safely.
 *
 * @param rebuiltDir - Base rebuild directory.
 * @param emptyDirectories - Directory paths to recreate.
 */
export async function restoreEmptyDirectories(
    rebuiltDir: string,
    emptyDirectories: string[],
): Promise<void> {
    const normalizedPath = normalizePath(rebuiltDir);

    for (const dir of emptyDirectories) {
        await mkdir(joinPath(normalizedPath, dir), {
            recursive: true,
        });
    }
}

/**
 * Splits one job's archive back into real files under `rebuiltDir`.
 *
 * @param outputDir - Directory containing the compiled archive files.
 * @param rebuiltDir - Directory to recreate original files into.
 * @param filename - The job's archive filename.
 * @returns The split outcome for this job.
 */
export async function splitArchiveFile(
    outputDir: string,
    rebuiltDir: string,
    filename: string,
): Promise<SplitFileOutcome> {
    const normalizedOutputPath = normalizePath(outputDir);
    const normalizedRebuiltPath = normalizePath(rebuiltDir);

    const resolvedPath = joinPath(normalizedOutputPath, filename);
    const archiveFile = Bun.file(resolvedPath);
    if (!(await archiveFile.exists())) {
        return {
            filename,
            archiveFound: false,
            filesRecreated: 0,
        };
    }

    const rawText = await archiveFile.text();
    const archivedFiles = parseArchive(rawText);

    let filesRecreated = 0;
    for (const archived of archivedFiles) {
        const normalizedArchivedPath = normalizePath(archived.originalPath);
        const writePath = joinPath(
            normalizedRebuiltPath,
            normalizedArchivedPath,
        );

        await mkdir(dirname(writePath), {
            recursive: true,
        });
    }
}

```

```

        await Bun.write(writePath, archived.content);

        filesRecreated++;
    }

    return {
        filename,
        archiveFound: true,
        filesRecreated,
    };
}

/**
 * Splits every job archive listed in `manifest.files`, in order, and
 * returns a full summary. Headless counterpart to views/SplitView.ts.
 *
 * @param outputDir - Directory containing the compiled archive files
 * and manifest.
 * @param rebuiltDir - Directory to recreate original files into.
 * @param hooks - Optional progress callbacks.
 * @returns The full {@link SplitSummary} for this run.
 */
export async function runSplit(
    outputDir: string,
    rebuiltDir: string,
    hooks?: SplitHooks,
): Promise<SplitSummary> {
    const normalizedOutputPath = normalizePath(outputDir);
    const normalizedRebuiltPath = normalizePath(rebuiltDir);

    const manifest = await readManifest(normalizedOutputPath);
    if (!manifest) {
        return {
            manifestFound: false,
            fileOutcomes: [],
            totalFilesRecreated: 0,
        };
    }

    if (hooks?.onSplitStart) {
        await hooks.onSplitStart(manifest);
    }

    await prepareRebuildDirectory(normalizedRebuiltPath);
    await restoreEmptyDirectories(
        normalizedRebuiltPath,
        manifest.emptyDirectories ?? [],
    );

    const fileOutcomes: SplitFileOutcome[] = [];

    for (const filename of Object.keys(manifest.files)) {
        if (hooks?.onFileStart) {
            await hooks.onFileStart(filename);
        }

        const outcome = await splitArchiveFile(
            normalizedOutputPath,
            normalizedRebuiltPath,
            filename,
        );
        fileOutcomes.push(outcome);
    }

```

```

        if (hooks?.onFileSuccess) {
            await hooks.onFileSuccess(filename, outcome);
        }
    }

    return {
        manifestFound: true,
        fileOutcomes,
        totalFilesRecreated: fileOutcomes.reduce(filesRecreatedReducer, 0),
    };
}

/**
 * Reads the manifest and prepares the rebuild directory (wipe +
 * restore empty dirs).
 *
 * @param outputDir - Directory containing the compiled archive files
 * and manifest.
 * @param rebuiltDir - Directory to recreate original files into.
 * @returns The read manifest, or `null` if `manifest.json` wasn't found.
 */
export async function initializeSplit(
    outputDir: string,
    rebuiltDir: string,
): Promise<Manifest | null> {
    const normalizedOutputPath = normalizePath(outputDir);
    const normalizedRebuiltPath = normalizePath(rebuiltDir);

    const manifest = await readManifest(normalizedOutputPath);
    if (!manifest) return null;

    await prepareRebuildDirectory(normalizedRebuiltPath);
    await restoreEmptyDirectories(
        normalizedRebuiltPath,
        manifest.emptyDirectories ?? [],
    );

    return manifest;
}

/**
 * Splits a single job's archive. A thin wrapper around {@link
 * splitArchiveFile}.
 *
 * @param outputDir - Directory containing the compiled archive files.
 * @param rebuiltDir - Directory to recreate original files into.
 * @param filename - The job's archive filename.
 * @returns The split outcome for this job.
 */
export async function splitSingleFile(
    outputDir: string,
    rebuiltDir: string,
    filename: string,
): Promise<SplitFileOutcome> {
    const normalizedOutputPath = normalizePath(outputDir);
    const normalizedRebuiltPath = normalizePath(rebuiltDir);

    return await splitArchiveFile(
        normalizedOutputPath,
        normalizedRebuiltPath,
        filename,
    );
}

```

```

const filesRecreatedReducer = (
  sum: number,
  outcome: SplitFileOutcome,
): number => {
  return sum + outcome.filesRecreated;
};

// ██████████< END >-██████████
// FILE NAME: : SplitService.ts
// TYPE      : .TS
// PATH      : packages/split/src/SplitService.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : Split.bin.ts
// TYPE      : .TS
// PATH      : packages/split/src/Split.bin.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// FILE-PATH: packages/split/src/Split.bin.ts
//
import type { ResolvedDeps, StandaloneExecution } from '@pendex/core';
import { Exit } from '@pendex/exit';
import { Split } from './Split';

/**
 * STANDALONE EXECUTION ENTRY POINT
 * `bun run packages/split/src/Split.bin.ts` —
 * only pulls a Config, no App/State required.
 */
const standaloneExecution: StandaloneExecution = async (): Promise<void> => {
  console.log();

  const { resolveRunnerDeps } = await import('@pendex/core');
  const state: ResolvedDeps = await resolveRunnerDeps();

  const split = new Split(state);
  await split.execute();

  const exit = new Exit({
    ...state,
    exit: process.exit.bind(process),
    exitCode: 0,
  });
  await exit.execute();
};

if (import.meta.main) {
  await standaloneExecution();
}

// ██████████< END >-██████████
// FILE NAME: : Split.bin.ts
// TYPE      : .TS
// PATH      : packages/split/src/Split.bin.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : cli.ts
// TYPE      : .TS

```

```

// PATH      : packages/utils/src/cli.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
/**
 * Copyright (C) 2026 Dustin Dew <phyxashun@gmail.com>
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU Affero General Public License as
 * published by the Free Software Foundation, either version 3 of the
 * license, or (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU Affero General Public License for more details.
 *
 * You should have received a copy of the GNU Affero General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 */

/**
 * @module cli
 *
 * The pendex CLI's process entry point: runs the interactive
 * {@link App} shell and reports any uncaught error as a fatal crash.
 */

import { App } from './components/App';
import { Message } from './components/Message';

/**
 * MAIN ENTRY POINT
 */
if (import.meta.main) {
  try {
    await App.run();
  } catch (err: unknown) {
    Message(err);
    process.exit(1);
  }
}

// ██████████< END >-██████████
// FILE NAME: : cli.ts
// TYPE      : .TS
// PATH      : packages/utils/src/cli.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/utils/src/index.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
export * from './components/App';
export * from './components/Message';
export * from './utils/errorNote';

// ██████████< END >-██████████
// FILE NAME: : index.ts
// TYPE      : .TS
// PATH      : packages/utils/src/index.ts

```

```

// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────██████████

// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
// FILE NAME: : Message.ts
// TYPE      : .TS
// PATH      : packages/utils/src/components/Message.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
export type MessageFn = (msg: unknown) => void;

/**
 * Logs a fatal, uncaught error to the console in a consistent format.
 *
 * @param msg - The error (or arbitrary thrown value) to report.
 */
export const Message: MessageFn = (msg: unknown): void => {
  if (msg instanceof Error) {
    console.error(`Fatal crash: ${msg.message}`);
  } else {
    const formattedMsg =
      typeof msg === 'object' && msg !== null
        ? JSON.stringify(msg)
        : String(msg);

    console.error(`Fatal crash: ${formattedMsg}`);
  }
};

// ██████████< END >────────────────────────────────────────────────────────────────────────────────██████████
// FILE NAME: : Message.ts
// TYPE      : .TS
// PATH      : packages/utils/src/components/Message.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >────────────────────────────────────────────────────────────────────────────────██████████

// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
// FILE NAME: : App.ts
// TYPE      : .TS
// PATH      : packages/utils/src/components/App.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────────────────────────────────────────────────────────────────────██████████
// FILE-PATH: packages/utils/src/components/App.ts
//
/**
 * @module App
 *
 * Single responsibility: wire up the interactive shell —load config,
 * build the command list, run the menu loop. This is the "root
 * component" in the React comparison: it mounts children (the commands)
 * and has almost no logic of its own, so —unlike Compile/Split—it's
 * NOT split into a separate service/view pair. Its only real behavior is
 * the select-loop, which is inherently both "state" (which command is
 * active) and "render" (the menu) at once; forcing those apart here would
 * add indirection without adding clarity. If that changes (e.g. the menu
 * grows real branching logic), split it the same way Compile/Split were.
 */

import { confirm, intro, isCancel, select } from '@clack/prompts';
import { Compile } from '@pendex/compile';
import type { Command, MainMenuOptions, State } from '@pendex/core';
import { resolveRunnerDeps, type ExitFn } from '@pendex/core';
import { Exit } from '@pendex/exit';

```

```

import { Split } from '@pendex/split';

/**
 * Clear the console before displaying the menu
 * Insert a blank line before displaying the menu
 */
const clear: () => void = (): void => {
    console.clear();
    console.log();
};

/**
 * Process-wide singleton that owns the interactive CLI shell: the
 * command list, the active {@link State}, and the select-loop that
 * drives the main menu. Use {@link Application.getInstance} to obtain
 * it; the constructor is private.
 */
export class Application {
    private static instance: Application | null = null;

    private readonly STRINGS = {
        title: ' Ⓢ PENDEX —PROJECT FILE CONSOLIDATOR ',
        mainMenu: 'Main Menu:',
        returnToMenu: 'Press Enter to return to main panel...',
        exit: 'exit',
    } as const;

    private _state!: State;
    private _commands: Command[] = [];
    private mainMenuOptions: MainMenuOptions[] = [];

    private constructor() {}

    /**
     * Create and/or return the singleton
     */
    public static getInstance(): Application {
        return this.instance ?? new Application();
    }

    /**
     * Reset the singleton
     * Useful for test isolation
     */
    public static resetInstance(): void {
        this.instance = null;
    }

    /**
     * The application's current state
     * (theme, config, category colors)
     */
    public get state(): State {
        return this._state;
    }

    /**
     * The available commands (Compile, Split, Exit) for this run.
     */
    public get commands(): Command[] {
        return this._commands;
    }
}

```

```

/**
 * Resolves runner deps and builds the command list and menu options.
 * Must be called (directly or via {@link run}) before
 * {@link runSingleIteration}.
 *
 * @param exitFn - Function used by the Exit command
 * to terminate the process.
 */
public async init(exitFn: ExitFn): Promise<void> {
    this._state = await resolveRunnerDeps();

    this._commands = [
        new Compile(this.state),
        new Split(this.state),
        new Exit({
            ...this.state,
            exit: exitFn,
            exitCode: 0,
        }),
    ];

    this.mainMenuOptions = this.commands.map(
        (cmd: Command): MainMenuOptions => ({
            value: cmd.key,
            label: cmd.label,
            hint: cmd.hint,
        }),
    );
}

public async returnToMenu(): Promise<boolean> {
    const wantsToReturn: symbol | boolean = await confirm({
        message: this.STRINGs.returnToMenu,
    });

    if (isCancel(wantsToReturn) || !wantsToReturn) {
        return false;
    }
    return true;
}

/**
 * Executes one cycle of the main application loop.
 * Returns `false` if the loop should terminate, `true` otherwise.
 * This method is public specifically for robust testability.
 */
public async runSingleIteration(): Promise<boolean> {
    clear();

    // Display app title
    intro(this.state.theme.title(this.STRINGs.title));

    // Display choices and get user's choice
    const choice: string | symbol = await select({
        message: this.state.theme.info(this.STRINGs.mainMenu),
        options: this.mainMenuOptions,
    });

    // Handle exit
    if (isCancel(choice) || choice === this.STRINGs.exit) {
        return false;
    }
}

```

```

    // Find the correct command based on the user's choice
    const command: Command | undefined = this.commands.find(
      (cmd: Command): boolean => cmd.key === choice,
    );

    // If a command is found, execute
    if (command) {
      clear();
      await command.execute();
    }
    return await this.returnToMenu();
  }

  /**
   * The main entry point for running the application.
   *
   * @param exitFn - Optional exit function (defaults to `process.exit`)
   */
  public async run(exitFn?: ExitFn): Promise<void> {
    await this.init(exitFn ?? process.exit.bind(process));

    while (await this.runSingleIteration()) {
      /**
       * This loop will continue as long as
       * runSingleIteration() returns true.
       */
    }

    // After the loop terminates, call the final exit command.
    const exitCmd: Exit = this.commands.find(
      (cmd: Command): cmd is Exit => cmd instanceof Exit,
    ) as Exit;

    if (exitCmd) await exitCmd.execute();
  }

  /**
   * Shared {@link Application} instance used as the CLI's composition root.
   */
  const App = Application.getInstance();
  export { App };

  // ██████████< END >-██████████
  // FILE NAME: : App.ts
  // TYPE      : .TS
  // PATH      : packages/utils/src/components/App.ts
  // PROCESSED: 2026-09-06T21:55:30.013Z
  // ██████████< END >-██████████

  // ██████████< START >-██████████
  // FILE NAME: : pause.ts
  // TYPE      : .TS
  // PATH      : packages/utils/src/Utils/pause.ts
  // PROCESSED: 2026-09-06T21:55:30.013Z
  // ██████████< START >-██████████

  export const pause = async (
    message = 'Press Enter to continue...',
  ): Promise<void> => {
    process.stdout.write(message);

    // Get a readable stream reader from Bun.stdin
    const reader = Bun.stdin.stream().getReader();

```

```
import process from 'node:process';
import type { Writable } from 'node:stream';
import { styleText } from 'node:util';

import { getColumns, settings } from '@clack/core';
```

```
import stringWidth from 'fast-string-width';
import { wrapAnsi } from 'fast-wrap-ansi';
```

```
const S_BAR = '┃';
const S_BAR_H = '─';
const S_CONNECT_LEFT = '├';
const S_CORNER_BOTTOM_LEFT = '└';
const S_CORNER_BOTTOM_RIGHT = '┘';
const S_CORNER_TOP_RIGHT = '┐';
const S_ERROR_SQUARE = '■';
```

```
type FormatFn = (line: string) => string;
export interface NoteOptions {
  output?: Writable;
  withGuide?: boolean;
  format?: FormatFn;
}
```

```
const defaultNoteFormatter = (line: string): string => line;
```

```
const wrapWithFormat = (
  message: string,
  width: number,
  format: FormatFn,
): string => {
  const opts = {
    hard: true,
    trim: false,
  };

  const wrapMsg = wrapAnsi(message, width, opts).split('\n');
  const maxWidthNormal = wrapMsg.reduce(
    (sum, ln) => Math.max(stringWidth(ln), sum),
    0,
  );

  const maxWidthFormat = wrapMsg
    .map(format)
```

```

        .reduce((sum, ln) => Math.max(stringWidth(ln), sum), 0);
const wrapWidth = width - (maxWidthFormat - maxWidthNormal);
return wrapAnsi(message, wrapWidth, opts);
};

export const errorNote = (
  message = '',
  title = 'Error Details',
  opts?: NoteOptions,
) => {
  const output: Writable = opts?.output ?? process.stdout;
  const hasGuide = opts?.withGuide ?? settings.withGuide;
  const format = opts?.format ?? defaultNoteFormatter;

  // Dynamically measure wrap length bounds against user's CLI width
  const wrapMsg = wrapWithFormat(message, getColumns(output) - 6, format);
  const lines = ['', ...wrapMsg.split('\n').map(format), ''];
  const titleLen = stringWidth(title);

  const len =
    Math.max(
      lines.reduce((sum, ln) => {
        const width = stringWidth(ln);
        return width > sum ? width : sum;
      }, 0),
      titleLen,
    ) + 2;

  const msg = lines
    .map(
      ln =>
        `${styleText('gray', S_BAR)} ${ln}${' '.repeat(len - stringWidth(ln))}${styleText('gray', S_BAR)}`,
    )
    .join('\n');

  const leadingBorder = hasGuide ? `${styleText('gray', S_BAR)}\n` : '';
  const bottomLeft = hasGuide ? S_CONNECT_LEFT : S_CORNER_BOTTOM_LEFT;

  output.write(
    `${leadingBorder}${styleText('red', S_ERROR_SQUARE)} ${styleText('red', title)} ${styleText(
      'gray',
      S_BAR_H.repeat(Math.max(len - titleLen - 1, 1)) +
      S_CORNER_TOP_RIGHT,
    )}\n${msg}\n${styleText('gray', bottomLeft + S_BAR_H.repeat(len + 2) + S_CORNER_BOTTOM_RIGHT)}\n`,
  );
};

// =====< END >-=====
// FILE NAME: : errorNote.ts
// TYPE      : .TS
// PATH      : packages/utils/src/utils/errorNote.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : validator.ts
// TYPE      : .TS
// PATH      : packages/utils/src/utils/validator.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
/**
 * @module validators
 *

```

```

* Small, pure input validators for interactive prompts (directory
* paths, glob-list strings). Each returns an error message string when
* invalid, or `undefined` when the input is acceptable —the shape
* `@clack/prompts`' `validate` option expects.
*/

/**
 * CONSTANTS
 */

/** User-facing validation error messages. */
const STRINGS = {
  required: 'This field is required and cannot be left blank.',
  noSpaces: 'Spaces are not allowed in directory path names.',
  invalidChars:
    'Directory name contains illegal path symbols (e.g., \\, :, *, ?, ", <, >, |).',
  emptyGlob: 'Glob lists cannot contain completely blank strings.',
} as const;

const SPACES_RE = /\s/;
const INVALID_CHARS_RE = /[<>:"|?*]/;
const EMPTY_GLOB_RE = /^(^s*,|s*,|,s*$)/;

/**
 * MAIN FUNCTIONS
 */

/**
 * Validates a directory path input: non-blank, no spaces, no illegal
 * path symbols.
 *
 * @param input - Raw user input to validate.
 * @returns An error message if invalid, or `undefined` if valid.
 */
export const validateDirectory = (input: string): string | undefined => {
  const trimmed = input.trim();
  if (!trimmed) return STRINGS.required;

  if (SPACES_RE.test(trimmed)) return STRINGS.noSpaces;
  if (INVALID_CHARS_RE.test(trimmed)) return STRINGS.invalidChars;

  return undefined;
};

/**
 * Validates a comma-separated glob-list input: non-blank, no
 * completely empty entries between commas.
 *
 * @param input - Raw user input to validate.
 * @returns An error message if invalid, or `undefined` if valid.
 */
export const validateGlobInput = (input: string): string | undefined => {
  const trimmed = input.trim();

  if (!trimmed) return STRINGS.required;
  if (EMPTY_GLOB_RE.test(trimmed)) return STRINGS.emptyGlob;

  return undefined;
};

// ██████████< END >-██████████
// FILE NAME: : validator.ts
// TYPE      : .TS

```

```
// PATH      : packages/utls/src/utls/validator.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// _____< END >_____

// _____< START >_____
// FILE NAME : one-text.ts
// TYPE      : .TS
// PATH      : utls/one-text.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// _____< START >_____

import { type Glob } from 'bun';

const INPUT_DIR = './ALL/';
const OUTPUT_FILE = './ALL/0_CONSOLIDATED_TEXT_FILES.txt';

async function consolidateFiles(): Promise<void> {
  try {
    // Initialize a built-in Bun Glob for text files
    const glob: Glob = new Bun.Glob('*.txt');

    const txtFiles: string[] = [];

    // Scan the target directory using Bun's built-in scanner
    for await (const filename of glob.scan({
      cwd: INPUT_DIR,
    })) {
      txtFiles.push(filename);
    }

    if (txtFiles.length === 0) {
      console.log('No .txt files found in the directory.');
```

```

}
// c8 ignore stop

// ██████████< END >-██████████
// FILE NAME: : one-text.ts
// TYPE      : .TS
// PATH      : utils/one-text.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : clean.ts
// TYPE      : .TS
// PATH      : utils/clean.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████

import { $, Glob } from 'bun';

/**
 * Logs a fatal, uncaught error to the console in a consistent format.
 *
 * @param msg - The error (or arbitrary thrown value) to report.
 */
export const Message = (msg: unknown): void => {
  if (msg instanceof Error) {
    console.error(`Fatal crash: ${msg.message}`);
  } else {
    console.error(`Fatal crash: ${msg}`);
  }
};

/**
 * Gracefully empties directory contents when the directory itself is
 * locked by Windows.
 *
 * Uses Bun-native shell execution which automatically handles OS differences.
 */
const emptyDirectoryContents = async (dirPath: string): Promise<void> => {
  try {
    // Bun Shell supports globbing and natively executes cleanups cross-platform
    await `$rm -rf ${dirPath}/*`;
  } catch (err) {
    console.error(`Could not empty locked directory ${dirPath}:`, err);
  }
};

const main = async () => {
  const nodeGlob = new Glob('**/node_modules');
  const bunGlob = new Glob('**/*.lock,lockb');

  // Using Bun's global "import.meta.dir" to ensure absolute execution paths
  const projectRoot = import.meta.dir || '.';

  console.log('Scanning for node_modules...');
  for await (const rawPath of nodeGlob.scan({
    cwd: projectRoot,
    onlyFiles: false,
  })) {
    console.log('Removing Directory: ', rawPath);

    try {
      // This moves the directory safely to your OS Recycle Bin/Trash
      await trash(rawPath);
    } catch (err: any) {

```

```

        // Handle locked directory fallback safely
        if (
            err.message?.includes('EACCES') ||
            err.message?.includes('permission denied')
        ) {
            console.warn(
                `⚠ Directory is locked by system. Attempting to empty contents of: ${rawPath}`,
            );
            await emptyDirectoryContents(rawPath);
        } else {
            console.error(`Failed to remove ${rawPath}:`, err);
        }
    }
}

console.log('Scanning for lockfiles...');
for await (const rawPath of bunGlob.scan({ cwd: projectRoot })) {
    console.log('Removing File: ', rawPath);
    try {
        await `$rm -f ${rawPath}`;
    } catch (err) {
        console.error(`Failed to remove ${rawPath}:`, err);
    }
}

console.log('🧹 Clean completed successfully!');
};

/**
 * MAIN ENTRY POINT
 */
// c8 ignore start
if (import.meta.main) {
    try {
        await main();
    } catch (err) {
        Message(err);
        process.exit(1);
    }
}
// c8 ignore stop

// ██████████< END >-██████████
// FILE NAME: : clean.ts
// TYPE      : .TS
// PATH      : utils/clean.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : HeaderComments.ts
// TYPE      : .TS
// PATH      : utils/HeaderComments.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >-██████████
// oxcint-disable typescript/no-explicit-any

/**
 * @module HeaderComments
 *
 * Standalone dev utility: injects or strips a `// FILE-PATH: <path>`
 * header comment across every matching project file. Runs as its own
 * interactive `@clack/prompts` session (`bun run src/utils/HeaderComments.ts`)

```

```

* with its own minimal `Command` shape and dark theme —deliberately
* decoupled from `@pendex/core`'s `Command`/theme system since this is a
* one-off maintenance tool, not part of the compile/split pipeline.
*/

```

```

import {
  cancel,
  confirm,
  intro,
  isCancel,
  log,
  note,
  outro,
  select,
  spinner,
} from '@clack/prompts';
import { Colors } from '@pendex/color';
import path from 'path';

```

```

/**
 * CONSTANTS
 */

```

```

/**
 * TYPES
 */

```

```

/** Minimal theme contract this standalone tool needs (a subset of
`@pendex/theme`'s `DefaultTheme`). */

```

```

interface Theme {
  primary(text: string): string;
  success(text: string): string;
  error(text: string): string;
  muted(text: string): string;
}

```

```

/** A fixed dark theme used when this tool runs standalone (no
`@pendex/theme` dependency). */

```

```

const DarkTheme: Theme = {
  primary: txt => Colors.bgCyan(Colors.black(` ${txt} `)),
  success: txt => Colors.green(txt),
  error: txt => Colors.red(txt),
  muted: txt => Colors.dim(txt),
};

```

```

/** Dependencies this tool needs: a theme and the subset of Config
relevant to file scanning. */

```

```

interface HeaderCommentsDeps {
  theme: Theme;
  config: {
    outputDir: string;
    exclude: string[];
  };
}

```

```

/** Result of attempting to add/remove a header comment on one file. */

```

```

interface HeaderUpdate {
  /** Whether the file's content was actually changed. */
  modified: boolean;
  /** Why nothing changed, when `modified` is `false`. */
  reason?: string;
}

```

```

/** Aggregate counts from a full inject/strip batch run. */
interface ScanSummary {
  /** Total files scanned. */
  total: number;
  /** Files actually modified. */
  processed: number;
  /** Files left unchanged (already correct, or excluded). */
  skipped: number;
  /** Files that errored during processing. */
  failed: number;
}

/** The menu choices this tool's interactive session offers. */
type HeaderAction = 'inject' | 'strip' | 'exit';

/**
 * MAIN CLASSES
 */

/** Minimal local command contract for this standalone tool
(independent of `@pendex/core`'s `Command`). */
export abstract class Command {
  abstract readonly key: string;
  abstract readonly label: string;
  abstract readonly hint: string;
  constructor(_deps: unknown) {}
  abstract execute(): Promise<void>;
}

/** Interactive tool that injects or strips `// FILE-PATH:` header
comments across the project. */
export class HeaderComments extends Command {
  readonly key = 'headercomments';
  readonly label = 'HeaderComments Manager';
  readonly hint = 'Inject or strip contextual path comment descriptors';

  private deps: HeaderCommentsDeps;

  private readonly BASE_DIR = process.cwd();
  private readonly COMMENT = '///';
  private readonly HEADER_COMMENT = `$$${this.COMMENT} FILE-PATH:`;
  private readonly SHEBANG_REGEX = /^#!.*/;
  private readonly FILE_EXTENSIONS = [
    'js',
    'ts',
    'jsx',
    'tsx',
    'json',
    'jsonc',
    'toml',
    'yaml',
    'yml',
    'md',
  ];

  private readonly BULLET = {
    arrow: '→',
    arrowHead: '➤',
    circle: {
      green: '●',
      white: '○',
      red: '●',
      purple: '●',
    },
  };

```

```

    },
    cancel: '✖',
    complete: '✅',
    success: '☑',
    error: '❌',
    exit: '✖',
    comment: '💬',
  } as const;

/**
 * @param deps - Theme and config subset this tool needs.
 */
constructor(deps: HeaderCommentsDeps) {
  super(deps);
  this.deps = deps;
}

/**
 * Converts a path to POSIX-style forward slashes.
 *
 * @param p - Path to convert.
 * @returns `p` with all backslashes replaced by forward slashes.
 */
private toUnixPath(p: string): string {
  return p.replace(/\\/g, '/');
}

/**
 * Whether a path should be skipped: inside `node_modules/.git`,
 * inside the compiled output directory, or matching a configured
 * exclude glob.
 *
 * @param filePath - Candidate path (relative or absolute).
 * @param compiledExcludes - Pre-compiled `Bun.Glob` exclude patterns.
 * @returns Whether the path should be ignored.
 */
private isIgnorePath(filePath: string, compiledExcludes: any[]): boolean {
  const unixPath = this.toUnixPath(filePath);
  return (
    unixPath.startsWith('node_modules/') ||
    unixPath.startsWith('.git/') ||
    unixPath.startsWith(`${this.deps.config.outputDir}/`) ||
    compiledExcludes.some(glob => glob.match(filePath))
  );
}

/**
 * Adds a `// FILE-PATH:` header comment to a file, unless one is
 * already present. Placed after a shebang line if the file starts
 * with one.
 *
 * @param absolutePath - Absolute path to the file to update.
 * @returns Whether the file was modified.
 */
public async enforceHeaderComments(
  absolutePath: string,
): Promise<HeaderUpdate> {
  const file = Bun.file(absolutePath);
  const contents = await file.text();

  if (contents.includes(this.HEADER_COMMENT)) {
    return {
      modified: false,
    };
  }
}

```

```

        reason: 'Header comment already present.',
    };
}

const relativePath = path.relative(this.BASE_DIR, absolutePath);
const unixRelativePath = this.toUnixPath(relativePath);
const targetHeader = `${this.HEADER_COMMENT} ${unixRelativePath}\n`;

const firstNewlineIdx = contents.indexOf('\n');
const firstLine =
    firstNewlineIdx === -1
        ? contents
        : contents.slice(0, firstNewlineIdx).replace('\r', '');

let updatedContent = '';
if (this.SHEBANG_REGEX.test(firstLine)) {
    const remainingContent =
        firstNewlineIdx === -1
            ? ''
            : contents.slice(firstNewlineIdx + 1);
    const separator = remainingContent.trim().length > 0 ? '\n' : '';
    updatedContent =
        `${firstLine}\n${targetHeader}${separator}${remainingContent}`;
} else {
    const separator = contents.length > 0 ? '\n' : '';
    updatedContent = `${targetHeader}${separator}${contents}`;
}

await Bun.write(absolutePath, updatedContent);
return { modified: true };
}

/**
 * Removes a `// FILE-PATH:` header comment from a file, if present.
 * Handles both shebang-prefixed and plain files.
 *
 * @param absolutePath - Absolute path to the file to update.
 * @returns Whether the file was modified.
 */
public async removeHeaderComments(
    absolutePath: string,
): Promise<HeaderUpdate> {
    const file = Bun.file(absolutePath);
    const contents = await file.text();

    if (!contents.includes(this.HEADER_COMMENT)) {
        return {
            modified: false,
            reason: 'Header comment not present.',
        };
    }

    const firstNewlineIdx = contents.indexOf('\n');
    const firstLine =
        firstNewlineIdx === -1
            ? contents
            : contents.slice(0, firstNewlineIdx).replace('\r', '');

    let updatedContent = '';

    if (this.SHEBANG_REGEX.test(firstLine)) {
        const remainingContent =
            firstNewlineIdx === -1

```

```

        ? ''
        : contents.slice(firstNewlineIdx + 1);
const nextNewlineIdx = remainingContent.indexOf('\n');
const secondLine =
    nextNewlineIdx === -1
        ? remainingContent
        : remainingContent.slice(0, nextNewlineIdx);

    if (secondLine.includes(this.HEADER_COMMENT)) {
        updatedContent = `${firstLine}\n${nextNewlineIdx ===
-1 ? '' : remainingContent.slice(nextNewlineIdx + 1)}';
    } else {
        updatedContent =
`${firstLine}\n${remainingContent.replace(this.HEADER_COMMENT, '')}`;
    }
    } else {
        if (firstLine.includes(this.HEADER_COMMENT)) {
            updatedContent =
                firstNewlineIdx === -1
                    ? ''
                    : contents.slice(firstNewlineIdx + 1);
        } else {
            updatedContent = contents.replace(
                new RegExp(
                    `.*${RegExp.escape(this.HEADER_COMMENT)}.*\n?`,
                    'g',
                    '',
                ),
                '',
            );
        }
    }
}

await Bun.write(absolutePath, updatedContent.trimStart());
return { modified: true };
}

/**
 * Scans every matching file under {@link BASE_DIR} and applies
 * `inject` or `strip` to each non-excluded file.
 *
 * @param action - Whether to inject or strip header comments.
 * @returns Aggregate counts for the run.
 */
private async processBatch(
    action: 'inject' | 'strip',
): Promise<ScanSummary> {
    const summary: ScanSummary = {
        total: 0,
        processed: 0,
        skipped: 0,
        failed: 0,
    };

    const extensionsList = this.FILE_EXTENSIONS.join(',');
    const glob = new Bun.Glob(`**/*.${extensionsList}`);

    const excludeGlobs = this.deps.config.exclude.map(
        ex => new Bun.Glob(ex),
    );

    const fileBuffer: string[] = [];

    for await (const relativePath of glob.scan({
        cwd: this.BASE_DIR,
    })) {

```

```

        fileBuffer.push(relativePath);
    }

    summary.total = fileBuffer.length;

    await Promise.all(
        fileBuffer.map(async relativePath => {
            if (this.isIgnorePath(relativePath, excludeGlobs)) {
                summary.skipped++;
                return;
            }

            const absolutePath = path.join(this.BASE_DIR, relativePath);
            try {
                const res =
                    action === 'inject'
                        ? await this.enforceHeaderComments(absolutePath)
                        : await this.removeHeaderComments(absolutePath);

                if (res.modified) summary.processed++;
                else summary.skipped++;
            } catch (error) {
                summary.failed++;
                log.error(
                    `${this.BULLET.error} Exception encountered
for ${relativePath}: ${error instanceof Error ? error.message :
String(error)}`,
                );
            }
        })
    );

    return summary;
}

/**
 * Runs the interactive inject/strip session: menu,
 * confirmation, batch run, and summary report.
 */
async execute(): Promise<void> {
    console.clear();
    intro(
        `${Colors.bgYellow(Colors.black(' HEADERCOMMENTS '))}
${Colors.yellow(Colors.dim('Scans and Synchronizes Headers'))}`,
    );

    const operation = await select<HeaderAction>({
        message: Colors.cyan('Pick an option:'),
        options: [
            {
                value: 'inject',
                label: 'Inject HeaderComments',
                hint: 'Append contextual tags',
            },
            {
                value: 'strip',
                label: 'Strip HeaderComments',
                hint: 'Scrub existing comment tags',
            },
            {
                value: 'exit',
                label: 'Exit menu context',
            },
        ],
    });

```

```

    ],
  });

  if (isCancel(operation) || operation === 'exit') {
    cancel(
      Colors.yellow(
        `${this.BULLET.success} Operation aborted. Project
files left unmodified.`,
      ),
    );
    return;
  }

  const shouldProceed = await confirm({
    message: `Verify and run structural updates across project
root directory?\n${this.BULLET.arrowHead}
(${Colors.cyan(this.BASE_DIR)})`,
    initialValue: operation === 'inject',
  });

  if (isCancel(shouldProceed) || !shouldProceed) {
    cancel(
      Colors.yellow(
        `${this.BULLET.success} Operation aborted. Project
files left unmodified.`,
      ),
    );
    return;
  }

  const s = spinner();
  s.start(
    Colors.cyan(
      'Evaluating active project workspaces and indexing targets...',
    ),
  );

  try {
    const metrics = await this.processBatch(operation);
    s.stop(
      Colors.green(
        'Header resolution sweep finished processing
successfully!!!\n',
      ),
    );
  };

  const labelText =
    this.BULLET.circle.green +
    (operation === 'inject' ? ' Processed:' : ' Stripped:');
  const hintText =
    this.BULLET.arrowHead +
    (operation === 'inject'
      ? ' Appended or confirmed targets'
      : ' Erased comment headers');

  const reportBody = [
    `${Colors.bold(labelText)}`,
    `${Colors.green(metrics.processed.toString().padStart(6))}`,
    `${Colors.dim(hintText)}`,

    `${Colors.bold(this.BULLET.circle.white + ' Skipped:
')} ${Colors.dim(metrics.skipped.toString().padStart(8))}`,
    `${Colors.dim(this.BULLET.arrowHead + ' Excluded folders & skips')}`,
  ];

```

```

        `${Colors.bold(this.BULLET.circle.red + ' Failed: ')}
${Colors.red(metrics.failed.toString().padStart(9))}
${Colors.dim(this.BULLET.arrowHead + ' Hard drive read/write
blocks')}}`,

        Colors.dim(
            '-----',
        ),

        `${Colors.bold(this.BULLET.circle.purple + ' TOTAL:
')} ${Colors.magenta(metrics.total.toString().padStart(10))}
${Colors.dim(this.BULLET.arrowHead + ' Aggregated project file
count')}}`,

        ].join('\n');

        note(reportBody, Colors.cyan(`Execution Metrics Summary Window`));
        outro(
            this.deps.theme.success(
                'Project workspace synchronization matches
verified state parameters.',
            ),
        );
    } catch (error: unknown) {
        s.stop(
            this.deps.theme.error(
                'A critical fatal failure crashed processing workflows.',
            ),
        );
        log.error(error instanceof Error ? error.message : String(error));
    }
}

/**
 * STANDALONE EXECUTION ENGINE ENTRY POINT
 */

// c8 ignore start
if (import.meta.main) {
    const app = new HeaderComments({
        theme: DarkTheme,
        config: {
            outputDir: 'ALL',
            exclude: [
                '**/*.toml,json,jsonc',
                '.vscode/**',
                'ALL/**',
                'ALL_REBUILT/**',
                'logs/**',
                'node_modules/**',
                'bun.lock',
                'package.json',
                'README.md',
                'tsconfig.json',
                'build/**',
                'dist/**',
                'coverage/**',
            ],
        },
    });

    app.execute().catch(err => {

```

```

        log.error(`Fatal crash: ${err?.message ?? err}`);
        process.exit(1);
    });
}
// c8 ignore stop

// =====< END >-=====
// FILE NAME: : HeaderComments.ts
// TYPE      : .TS
// PATH      : utils/HeaderComments.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-=====

// =====< START >-=====
// FILE NAME: : string-extension.ts
// TYPE      : .TS
// PATH      : utils/string-extension.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-=====
export const { Colors } = await import('@pendex/color');

export const SEPARATOR = '/';
export const PATH_SEPARATOR_REGEX = /\\/g;
export const CONSECUTIVE_SLASH_REGEX = /(?!^)(?!\\b[a-zA-Z]{2,})\\{2,}/g;

export type Segment = string | number;
export type PathSegment = Segment | false | undefined | null;

// Declare as new String method
declare global {
    interface String {
        toPosixPath(): string;
        normalizePath(): string;
        joinPath(...segments: PathSegment[]): string;
        label(): string;
        red(): string;
        green(): string;
    }
}

/**
 * Converts a path to POSIX-style forward slashes.
 *
 * @param this - Path to convert.
 * @returns `this` with all backslashes replaced by forward slashes.
 */
String.prototype.toPosixPath = function (this: string): string {
    return this.replace(PATH_SEPARATOR_REGEX, SEPARATOR);
};

/**
 * Normalizes all slashes to a standard web-safe format.
 * Exported so other modules can validate untrusted raw config string inputs.
 *
 * @param this - Path to convert.
 * @returns `this` with all double forward slashes replaced by single
 * forward slashes.
 */
String.prototype.normalizePath = function (this: string): string {
    return this.toPosixPath().replace(CONSECUTIVE_SLASH_REGEX, SEPARATOR);
};

/**

```

```

* Joins path segments with a single SEPARATOR, collapsing accidental doubles.
* Adjusted signature to safely accept and drop falsy conditional values.
*
* @param segments - Path segments to join; falsy segments are dropped.
* @returns The joined path.
*/

```

```

String.prototype.joinPath = function (...segments: PathSegment[]): string {
    const allSegments = [this, ...segments].filter(Boolean) as Segment[];

    // Convert each segment to a posix path
    const posixPath = allSegments
        .map(segment => String(segment).toPosixPath())
        .join(SEPARATOR);

    // Normalize entire combined string
    return posixPath.normalizePath();
};

```

```

String.prototype.label = function (this: string): string {
    return Colors.cyan(this);
};

```

```

String.prototype.red = function (this: string): string {
    return Colors.red(this);
};

```

```

String.prototype.green = function (this: string): string {
    return Colors.green(this);
};

```

```

// ██████████ < END > - ██████████
// FILE NAME: : string-extension.ts
// TYPE      : .TS
// PATH      : utils/string-extension.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < END > - ██████████

```

```

// ██████████ < START > - ██████████
// FILE NAME: : app.strings.ts
// TYPE      : .TS
// PATH      : utils/app.strings.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████ < START > - ██████████

```

```

export const STRINGS = {
    App: {
        title: ' @ PENDIX —PROJECT FILE CONSOLIDATOR ',
        mainMenu: 'Main Menu:',
        returnToMenu: 'Press Enter to return to main panel...',
        exit: 'exit',
    },
    Compile: {
        key: 'compile',
        label: 'Compile project',
        hint: 'Consolidates project files',
    },
    Exit: {
        key: 'exit',
        label: 'Exit program',
        hint: 'Terminate system process',
    },
    ExitView: {
        goodbye: '👋 Exiting @pendex. System process ended.',
    },
};

```

```
} as const;
```

```
// ██████████< END >────────────────██████████  
// FILE NAME: : app.strings.ts  
// TYPE      : .TS  
// PATH      : utils/app.strings.ts  
// PROCESSED: 2026-09-06T21:55:30.013Z  
// ██████████< END >────────────────██████████
```

```
// ██████████< START >────────────────██████████  
// FILE NAME: : update-config.ts  
// TYPE      : .TS  
// PATH      : utils/update-config.ts  
// PROCESSED: 2026-09-06T21:55:30.013Z  
// ██████████< START >────────────────██████████
```

```
/**  
 * @module update-config  
 *  
 * Standalone dev utility: scans the current project for file  
 * extensions in use, buckets them into Pendex's standard job  
 * categories, and writes a fresh `src/config/jobs.toml`. Meant to be  
 * run once against a new project to bootstrap a starting config, not  
 * as part of the compile/split pipeline.  
 */
```

```
import { Glob } from 'bun';  
import { extname } from 'path';
```

```
/**  
 * Paths this tool reads from and writes to.  
 */
```

```
const Constants = {  
  GITIGNORE_PATH: '.gitignore',  
  OUT_PATH: 'src/config/jobs.toml',  
};
```

```
/**  
 * Extension/pattern lists used to bucket discovered  
 * files into job categories.  
 */
```

```
const CATEGORIES = {  
  SOURCE_FILES: [  
    '.js',  
    '.jsx',  
    '.mjs',  
    '.cjs',  
    '.ts',  
    '.tsx',  
    '.mts',  
    '.cts',  
    '.vue',  
    '.svelte',  
    '.astro',  
    '.py',  
    '.pyw',  
    '.sh',  
    '.bash',  
    '.ps1',  
    '.c',  
    '.h',  
    '.cpp',  
    '.hpp',  
    '.cc',
```



```
    '.scss',
    '.sass',
    '.less',
    '.styl',
    '.pcss',
    '.sss',
    '.wxss',
    '.acss',
],
CONFIG_FILES: [
    '.json',
    '.jsonc',
    '.yaml',
    '.yml',
    '.toml',
    '.config.*',
    'config.*',
    '.editorconfig',
    '.browserslistrc',
    'browserslist',
    '*ignore',
    '.gitattributes',
    '.gitmodules',
    '.env*',
    'Dockerfile*',
    'docker-compose*.yml',
    '.github/workflows/*.yml',
    'package.json',
    'package-lock.json',
    'yarn.lock',
    'pnpm-lock.yaml',
    'bun.lockb',
    'composer.json',
    'composer.lock',
    'Cargo.toml',
    'Cargo.lock',
    'Gemfile',
    'Gemfile.lock',
    '.eslintrc*',
    'eslint.config.*',
    '.prettierrc*',
    'prettier.config.*',
    '.stylelintrc*',
    'stylelint.config.*',
    'tsconfig*.json',
    '.babelrc*',
    'babel.config.*',
    'webpack.config.*',
    'vite.config.*',
    'rollup.config.*',
    'next.config.*',
    'nuxt.config.*',
    'gatsby-config.*',
    'gulpfile.*',
    '*/settings.json',
    '.vscode/**/*.json',
    '.idea/**/*.xml',
],
TEST_FILES: [
    '*.test.*',
    '*.spec.*',
    '*_test.*',
    '*_spec.*',
```

```

    '*-test.*',
    'test_.*.*',
    'test.*',
    'test/**/*',
    'test_.py',
    '*Test.php',
    '*Spec.scala',
    '.__tests_/**/*',
    '.__snapshots_/**/*',
    '.__mocks_/**/*',
    'tests/**/*',
    'specs/**/*',
    'jest.config.*',
    'vitest.config.*',
    'playwright.config.*',
    'cypress.config.*',
    'karma.conf.*',
    'setupTests.*',
    'test-setup.*',
    'test.setup.*',
  ],
  DOC_FILES: [
    'README*',
    'readme*',
    'LICENSE*',
    'license*',
    'COPYING*',
    'NOTICE*',
    'CHANGELOG*',
    'changelog*',
    'HISTORY*',
    'RELEASES*',
    '.md',
    '.markdown',
    '.mdx',
    '.rst',
    '.adoc',
    '.asciidoc',
    '.txt',
    'CONTRIBUTING*',
    'contributing*',
    'CODE_OF_CONDUCT*',
    'code_of_conduct*',
    'SECURITY*',
    'SUPPORT*',
    'docs/**/*',
    'doc/**/*',
    '.github/ISSUE_TEMPLATE/**/*',
    '.github/PULL_REQUEST_TEMPLATE/**/*',
  ],
},
};

/** Glob patterns excluded from every scan performed by this tool. */
const GLOBAL_EXCLUDES = [
  '**/node_modules/**',
  '**/.git/**',
  '**/dist/**',
  '**/build/**',
  '**/*.lockb',
  '**/*.lock',
  '**/package-lock.json',
  '**/*.map',
  '**/*.png',

```

```

    '**/*.jpg',
    '**/*.ico',
    '**/*.gif',
    '**/*.mp4',
    '**/*.mp3',
    '**/*.pnpm-lock.yaml',
    '**/*.min.js',
    '**/*.js.map',
    '**/*.min.css',
    'coverage/**/*',
    'node_modules/**/*',
    'ALL/**/*',
    '**/.env',
    '*/ALL/**/*',
    '*/ALL_REBUILT/**/*',
    '**/.stylelintrc*',
    '**/stylelint.config.*',
    '**/__snapshots__/**/*',
    '**/__mocks__/**/*',
];

/**
 * Scans the project, categorizes discovered file extensions,
 * and writes a generated `jobs.toml`.
 */
class ConfigUpdater {
  private gitignore: string[] = [];
  private foundExtensions: Set<string> = new Set();

  /**
   * Convert glob-style gitignore patterns to simple regex for basic filtering
   *
   * @param filePath - Candidate path to check.
   * @returns Whether the path is ignored
   * (matches `.gitignore` or a hardcoded exclusion).
   */
  private isIgnored(filePath: string): boolean {
    // Quick check against standard hardcoded exclusions
    if (filePath.includes('node_modules/') || filePath.includes('.git/'))
      return true;

    for (const pattern of this.gitignore) {
      const cleanPattern = pattern.replace(/\//, '').replace(/\$/ , '');
      if (filePath.includes(cleanPattern)) {
        return true;
      }
    }
    return false;
  }

  /**
   * Loads `.gitignore` patterns (if present) into {@link gitignore}.
   */
  private async loadGitignore(): Promise<void> {
    const gitignoreFile = Bun.file(Constants.GITIGNORE_PATH);
    if (await gitignoreFile.exists()) {
      const content = await gitignoreFile.text();
      this.gitignore = content
        .split(/\r?\n/)
        .filter(line => line.trim() && !line.startsWith('#'));
    }
  }
}

```

```

/**
 * Scans every file under `process.cwd()` and records its
 * extension (or test-file pattern) into {@link foundExtensions}.
 */
private async scanFiles(): Promise<void> {
  const glob = new Glob('**/*');

  // Bun.Glob is insanely fast for scanning directories
  for await (const file of glob.scan({
    cwd: process.cwd(),
    onlyFiles: true,
  })) {
    if (this.isIgnored(file)) continue;

    const isTestFile = CATEGORIES.TEST_FILES.some(testExt =>
      file.endsWith(testExt),
    );

    if (isTestFile) {
      const testExt = CATEGORIES.TEST_FILES.find(ext =>
        file.endsWith(ext),
      );
      this.foundExtensions.add(testExt);
    } else {
      const ext = extname(file).toLowerCase();
      if (ext) this.foundExtensions.add(ext);
    }
  }
}

/**
 * Builds the full `jobs.toml` content from every discovered extension,
 * bucketed into categories and formatted as job entries.
 *
 * @returns The generated TOML text.
 */
private generateTOML(): string {
  const allFoundExts = Array.from(this.foundExtensions);
  const jobMappings: Record<string, string[]> = {
    SOURCE_FILES: [],
    WEB_FILES: [],
    STYLE_FILES: [],
    TERMINAL_FILES: [],
    CONFIG_FILES: [],
    DOC_FILES: [],
    TEST_FILES: [],
    MISC_FILES: [],
  };

  // Categorize found extensions
  for (const ext of allFoundExts) {
    let categorized = false;
    for (const [category, extensions] of Object.entries(CATEGORIES)) {
      if (extensions.includes(ext)) {
        jobMappings[category]!.push(`**/*${ext}`);
        categorized = true;
        break;
      }
    }
    if (!categorized) {
      jobMappings.MISC_FILES!.push(`**/*${ext}`);
    }
  }
}

```

```

// Build the TOML string manually
let tomlString = `theme = "dark"\n\noutputDir = "ALL"\nrebuiltDir = "ALL_REBUILT"\n\n`;

// Format Global Excludes
tomlString += `exclude = [\n`;
tomlString += GLOBAL_EXCLUDES.map(ex => ` "${ex}"`).join(',\n');
tomlString += `\n]\n\n`;

const orderedJobNames = [
  'SOURCE_FILES',
  'WEB_FILES',
  'STYLE_FILES',
  'TERMINAL_FILES',
  'CONFIG_FILES',
  'DOC_FILES',
  'TEST_FILES',
  'MISC_FILES',
];

let index = 1;
for (const jobName of orderedJobNames) {
  const includeGlobs = jobMappings[jobName];

  const excludeGlobs = allFoundExts
    .filter(ext => !includeGlobs!.includes(`**/*${ext}`))
    .map(ext => `**/*${ext}`);

  const description = `Includes files matching: ${includeGlobs!.length > 0 ? includeGlobs!.join(', ') : 'None'}. Excludes all other
project extensions.`;

  tomlString += `[[jobs]]\n`;
  tomlString += `filename = "${index}_${jobName}.txt"\n`;
  tomlString += `description = "${description}"\n`;

  tomlString += `include = [\n`;
  if (includeGlobs!.length > 0)
    tomlString +=
      includeGlobs!.map(inc => ` "${inc}"`).join(',\n') + `\n`;
  tomlString += `]\n`;

  tomlString += `exclude = [\n`;
  if (excludeGlobs.length > 0)
    tomlString +=
      excludeGlobs.map(ex => ` "${ex}"`).join(',\n') + `\n`;
  tomlString += `]\n\n`;

  index++;
}

return tomlString.trim() + '\n';
}

/**
 * Orchestrates a full run: load `.gitignore`, scan files,
 * generate TOML, and write it to {@link Constants.OUT_PATH}.
 */
public async run(): Promise<void> {
  console.log('Loading .gitignore...');
  await this.loadGitignore();

  console.log('Scanning directories using Bun.Glob...');
  await this.scanFiles();

```



```

    expectedNormalized: 'src/components/buttons/SubmitButton.tsx',
  },
  {
    name: 'Consecutive Slashes (Internal)',
    input: 'folder/subfolder//nested///file.txt',
    expectedPosix: 'folder/subfolder//nested///file.txt',
    expectedNormalized: 'folder/subfolder/nested/file.txt',
  },
  {
    name: 'Windows UNC Network Share',
    input: '\\\\server\\share\\folder',
    expectedPosix: '//server/share/folder',
    expectedNormalized: '//server/share/folder',
  },
  {
    name: 'Web Protocol (HTTP)',
    input: 'http://example.com',
    expectedPosix: 'http://example.com',
    expectedNormalized: 'http://example.com',
  },
  {
    name: 'Windows Drive Letter with Double Slash',
    input: 'C:\\\\folder\\\\subfolder',
    expectedPosix: 'C://folder//subfolder',
    expectedNormalized: 'C:/folder/subfolder',
  },
  {
    name: 'Trailing Slashes',
    input: 'usr/local/bin/',
    expectedPosix: 'usr/local/bin/',
    expectedNormalized: 'usr/local/bin/',
  },
  {
    name: 'joinPath: Standard Append',
    input: 'src',
    segments: ['components', 'Button.tsx'],
    expectedPosix: 'src/components/Button.tsx',
    expectedNormalized: 'src/components/Button.tsx',
  },
  {
    name: 'joinPath: Falsy Conditions & Numbers',
    input: 'src',
    // oxlint-disable-next-line no-constant-binary-expression
    segments: ['v1', false && 'premium', 'users', 404],
    expectedPosix: 'src/v1/users/404',
    expectedNormalized: 'src/v1/users/404',
  },
  {
    name: 'joinPath: Mixed Backslashes in Segments',
    input: 'src',
    segments: ['assets\\images', 'logo.png'],
    expectedPosix: 'src/assets/images/logo.png',
    expectedNormalized: 'src/assets/images/logo.png',
  },
  /**
   * PURPOSEFULLY FAILING TESTS
   */
  {
    name: 'Intentional Failure: Incorrect Mapped Path',
    input: 'C:\\Users\\Guest',
    expectedPosix: 'C:/Wrong/Path/Here',
    expectedNormalized: 'C:/Wrong/Path/Here',
  },
},

```

```

    {
      name: 'Intentional Failure: joinPath Output Mismatch',
      input: 'dist',
      segments: ['bundles', 'app.js'],
      expectedPosix: 'dist/wrong/app.js',
      expectedNormalized: 'dist/wrong/app.js',
    },
  ],

  console.log();
  intro(
    Colors.bgYellow(Colors.black(Colors.bold('STRING EXTENSIONS TESTING'))),
  );

  const deferredFailures: any[] = [];

  // Process tasks sequentially to retain full control
  // over failures without process abandonment
  for (const [index, test] of testCases.entries()) {
    const actualPosix = test.segments
      ? test.input.joinPath(...test.segments)
      : test.input.toPosixPath();

    const actualNormalized = test.segments
      ? actualPosix
      : test.input.normalizePath();

    const posixPassed = actualPosix === test.expectedPosix;
    const normalizedPassed = actualNormalized === test.expectedNormalized;

    const passedTag = Colors.green('Passed');
    const failedTag = Colors.red('Failed');
    const testNumber = Colors.cyan(`Test #${index + 1}`);
    const testName = Colors.magenta(`${test.name}`);

    if (!posixPassed || !normalizedPassed) {
      deferredFailures.push({
        name: test.name,
        input: test.input,
        segments: test.segments,
        posix: !posixPassed,
        normalized: !normalizedPassed,
        expectedPosix: test.expectedPosix,
        actualPosix,
        expectedNormalized: test.expectedNormalized,
        actualNormalized,
      });

      log.error(`${failedTag}...${testNumber}: ${testName}`);
    } else {
      log.success(`${passedTag}...${testNumber}: ${testName}`);
    }
  }

  const completed = () => {
    const complete = '✅ All path extension tests passed successfully!';
    outro(Colors.green(Colors.bold(complete)));
  };

  try {
    if (deferredFailures.length > 0) {
      throw new Error(JSON.stringify(deferredFailures));
    }
  }

```

```

    completed();
  } catch (error: any) {
    const errorTitle = 'Test Suite Completed with Failures: ';
    log.error(Colors.bgRed(Colors.black(Colors.bold(errorTitle))));

    let failuresToPrint: any[];

    try {
      failuresToPrint = JSON.parse(error.message);
    } catch {
      console.error(error);
      outro(Colors.red('Fatal infrastructure failure occurred. '));
      process.exit(1);
    }

    for (const details of failuresToPrint) {
      const lines: [string, string][] = [
        ['Input'.label(), `${details.input}`],
      ];

      if (details.segments) {
        lines.push([
          'Segments'.label(),
          JSON.stringify(details.segments),
        ]);
      }

      if (details.posix) {
        lines.push([
          'Expected Posix'.label(),
          `${details.expectedPosix}`,
        ]);
        lines.push([
          'Got Posix'.label(),
          `${details.actualPosix}`.red(),
        ]);
      }

      if (details.normalized) {
        lines.push([
          'Expected Normalized'.label(),
          `${details.expectedNormalized}`,
        ]);
        lines.push([
          'Got Normalized'.label(),
          `${details.actualNormalized}`.red(),
        ]);
      }
    }

    const maxLength = Math.max(...lines.map(([label]) => label.length));
    const noteContent = lines
      .map(
        ([label, value]) => `${label.padEnd(maxLength)} : ${value}`,
      )
      .join('\n');

    errorNote(
      noteContent,
      Colors.red(`Failure Details: ${details.name}`),
    );
  }

  log.error(Colors.red('Process terminated due to test failures. '));
}

```

```

    completed();
  };

  // Execute the tests
  export const runTests = async () => {
    try {
      await testPathExtensions();
      console.log('TESTING COMPLETE!!!');
      process.exit(0);
    } catch (err: any) {
      console.error(err);
      process.exit(1);
    }
    process.exit(0);
  };
};

//runTests();

// DEBUG END

// =====< END >-
// FILE NAME: : string-extension-test.ts
// TYPE      : .TS
// PATH      : utils/string-extension-test.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< END >-

// =====< START >-
// FILE NAME: : ToPdf.ts
// TYPE      : .TS
// PATH      : utils/ToPdf.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// =====< START >-
// FILE-PATH: src/utils/ToPdf.ts
//
import fontkit from '@pdf-lib/fontkit';
import {
  PageSizes,
  PDFDocument,
  PDFFont,
  PDFPage,
  rgb,
  type Color,
} from 'pdf-lib';

import type { Bunfile } from 'bun';
import path from 'path';

import Prism, { Token } from 'prismjs';
import loadLanguages from 'prismjs/components/index.js';

export interface PDFDimensions {
  width: number;
  height: number;
}

export type TextWidth = (text: string) => number;

export interface Config {
  fontSize?: number;
  lineHeight?: number;
  marginTop?: number;
  marginBottom?: number;
}

```

```

marginLeft?: number;
marginRight?: number;
fontFile?: string;
textColor?: Color;
textOpacity?: number;
pageSize?: [number, number] | keyof typeof PageSizes;
pageOrientation?: 'portrait' | 'landscape';
wordWrap?: 'word' | 'char' | 'none';
syntaxHighlighting?: boolean;
language?: string;
backgroundColor?: Color;
// New option to force everything into a single, continuous vertical canvas page
oneLongPage?: boolean;
}

```

```

export const DEFAULT_CONFIG: Required<Config> = {
  fontSize: 8,
  lineHeight: 12,
  marginTop: 50,
  marginBottom: 50,
  marginLeft: 50,
  marginRight: 50,
  fontFile:
    './assets/IosevkaNerdFontMono-Regular-stripped.ttf',
  textColor: rgb(0.96, 0.96, 0.94),
  textOpacity: 1.0,
  pageSize: 'Letter',
  pageOrientation: 'portrait',
  wordWrap: 'word',
  syntaxHighlighting: true,
  language: 'typescript',
  backgroundColor: rgb(0.11, 0.12, 0.13),
  oneLongPage: false,
};

```

```

export const SYNTAX_THEME: Record<string, Color> = {
  keyword: rgb(0.97, 0.15, 0.45),
  string: rgb(0.65, 0.86, 0.4),
  comment: rgb(0.46, 0.48, 0.42),
  number: rgb(0.68, 0.51, 1.0),
  function: rgb(0.4, 0.85, 0.94),
  operator: rgb(0.97, 0.61, 0.12),
  punctuation: rgb(0.8, 0.8, 0.8),
  boolean: rgb(0.97, 0.61, 0.12),
  classname: rgb(0.4, 0.85, 0.94),
  plain: rgb(0.96, 0.96, 0.94),
};

```

```

export const NEWLINE: RegExp = /\r?\n/;

```

```

interface TextAtom {
  text: string;
  color: Color;
}

```

```

interface RenderableWrappedLine {
  atoms: TextAtom[];
  textStr: string;
}

```

```

// oxlint-disable-next-line typescript/no-duplicate-type-constituents
type ParseTokenFn = (token: string | Token, type?: string) => void;

```

```

export class ToPdfService {
  private readonly config: Required<Config>;

  constructor(config: Required<Config>) {
    this.config = config;
    if (this.config.syntaxHighlighting) {
      try {
        loadLanguages([this.config.language]);
      } catch (e: unknown) {
        if (e instanceof Error) {
          console.warn(
            `⚠ [ToPdf Warning]: Could not load Prism definition for "${this.config.language}". Fallback 'clike' styling will be used.
            Error details: ${e.message}`,
          );
        } else {
          console.warn(
            `⚠ [ToPdf Warning]: Could not load Prism definition for "${this.config.language}". An unexpected error occurred. Fallback
            'clike' styling will be used.`,
          );
        }
      }
    }
  }

  // Adjusted to accept a dynamic custom height modifier parameter overrides
  private createPageWithLayout(
    pdfDoc: PDFDocument,
    customHeight?: number,
  ): PDFPage {
    // Destructure into explicit independent primitives to ensure no array reference mutations
    let [baseWidth, baseHeight] =
      typeof this.config.pageSize === 'string'
        ? PageSizes[this.config.pageSize]
        : this.config.pageSize;

    // Override vertical sizing immediately if computing a single long page strip
    if (customHeight !== undefined) {
      baseHeight = customHeight;
    }

    // Handle structural landscape orientation transformation BEFORE instantiating the canvas
    if (
      this.config.pageOrientation === 'landscape' &&
      customHeight === undefined
    ) {
      const temp: number = baseWidth;
      baseWidth = baseHeight;
      baseHeight = temp;
    }

    // Safely generate a flawlessly dimensioned canvas layout frame up front
    const page: PDFPage = pdfDoc.addPage([baseWidth, baseHeight]);
    const { width, height } = page.getSize();

    // Draw the solid background color wrapper safely matching the canvas bounds
    page.drawRectangle({
      x: 0,
      y: 0,
      width: width,
      height: height,
      color: this.config.backgroundColor,
    });
  }
}

```

```

    return page;
}

public async convertToPdf(
    sourcePath: string,
    outputPath: string,
    fontPath: string = this.config.fontFile,
): Promise<void> {
    const textContent: string = await this.getText(sourcePath);
    const pdfDoc: PDFDocument = await PDFDocument.create();
    pdfDoc.registerFontkit(fontkit);

    const font: PDFFont = await this.getFontSafely(fontPath, pdfDoc);

    // Determine target content layout bounding widths up front
    let initialPageWidth: number;
    if (typeof this.config.pageSize === 'string') {
        initialPageWidth = PageSizes[this.config.pageSize][0];
    } else {
        initialPageWidth = this.config.pageSize[0];
    }
    if (
        this.config.pageOrientation === 'landscape' &&
        !this.config.oneLongPage
    ) {
        if (typeof this.config.pageSize === 'string') {
            initialPageWidth = PageSizes[this.config.pageSize][1];
        } else {
            initialPageWidth = this.config.pageSize[1];
        }
    }

    const maxLineWidth: number =
        initialPageWidth -
        (this.config.marginLeft + this.config.marginRight);

    /**
     * PRE-PROCESS TOKENS
     */
    const renderingLines: TextAtom[][] = [[]];
    let currentLineIdx: number = 0;

    // Normalize tabs globally up front to ensure character counts and widths match perfectly
    const cleanTextContent: string = textContent.replace(/\t/g, ' ');

    if (this.config.syntaxHighlighting) {
        const grammar =
            Prism.languages[this.config.language] || Prism.languages.clike;
        const globalTokens: (string | Token)[] = Prism.tokenize(
            cleanTextContent,
            grammar!,
        );

        const parseToken: ParseTokenFn = (
            token: string | Prism.Token,
            type?: string,
        ): void => {
            if (typeof token === 'string') {
                const lines: string[] = token.split(/\r?\n/);
                const tokenType: string = type ?? 'plain';
                const activeColor: Color =
                    SYNTAX_THEME[tokenType] ?? this.config.textColor;
            }
        };
    }

```

```

        lines.forEach((lineText, index): void => {
            if (index > 0) {
                currentLineIdx++;
                renderingLines[currentLineIdx] = [];
            }
            if (lineText.length > 0) {
                renderingLines[currentLineIdx]!.push({
                    text: lineText,
                    color: activeColor,
                });
            }
        });
    } else {
        if (Array.isArray(token.content)) {
            token.content.forEach(subToken =>
                parseToken(subToken, token.type),
            );
        } else {
            parseToken(token.content, token.type);
        }
    }
};

globalTokens.forEach(token => parseToken(token));
} else {
    const rawLines: string[] = cleanTextContent.split(/\r?\n/);
    rawLines.forEach((line, index): void => {
        renderingLines[index] = [
            { text: line, color: this.config.textColor },
        ];
    });
}

/**
 * PRE-WRAPPING CALCULATIONS & MEASUREMENTS LAYER
 */
const allFinalWrappedLines: RenderableWrappedLine[] = [];
const textWidthFn: TextWidth = (text: string): number =>
    font.widthOfTextAtSize(text, this.config.fontSize);

for (const rawLineAtoms of renderingLines) {
    const fullLineText: string = rawLineAtoms
        .map(atom => atom.text)
        .join('');

    // Get wrapped versions of strings
    const wrappedLineStrings: string[] = this.wrapText(
        fullLineText,
        maxLineWidth,
        textWidthFn,
    );

    let atomIndex: number = 0;
    let atomCharOffset: number = 0;

    for (const wrappedString of wrappedLineStrings) {
        const lineAtoms: TextAtom[] = [];

        // Reconstruct token highlighting context based on exactly what text wrap chose
        // Strip out structural wrap-injected space variants if wrapText shifts characters
        for (let i: number = 0; i < wrappedString.length; i++) {
            if (atomIndex >= rawLineAtoms.length) break;

            let currentAtom = rawLineAtoms[atomIndex];

```

```

    // Safety logic: If wrap-injected characters (like added spaces) fall out of bounds
    // with original source string layout, dynamically bridge using current token style
    if (atomCharOffset >= currentAtom!.text.length) {
        atomIndex++;
        atomCharOffset = 0;
        if (atomIndex >= rawLineAtoms.length) break;
        currentAtom = rawLineAtoms[atomIndex];
    }

    const charToMatch = wrappedString[i];
    const originalChar = currentAtom!.text[atomCharOffset];

    // If characters align, safely build or extend current colored chunk
    if (charToMatch === originalChar) {
        const lastAtom = lineAtoms[lineAtoms.length - 1];
        if (lastAtom && lastAtom.color === currentAtom!.color) {
            lastAtom.text += charToMatch;
        } else {
            lineAtoms.push({
                text: charToMatch!,
                color: currentAtom!.color,
            });
        }
        atomCharOffset++;
    } else {
        // Fallback for wrapped-injected layout white-space
        const lastAtom = lineAtoms[lineAtoms.length - 1];
        const fallbackColor: Color = lastAtom
            ? lastAtom.color
            : this.config.textColor;
        if (lastAtom) {
            lastAtom.text += charToMatch;
        } else {
            lineAtoms.push({
                text: charToMatch!,
                color: fallbackColor,
            });
        }
    }
}

allFinalWrappedLines.push({
    atoms: lineAtoms,
    textStr: wrappedString,
});
}

/**
 * DYNAMIC CANVAS VS PAGINATION DISPATCH ROUTING
 */
let page: PDFPage;
let currentY: number;
let pageHeight: number;
const pageBottomThreshold: number = this.config.marginBottom;

if (this.config.oneLongPage) {
    const contentHeight: number =
        allFinalWrappedLines.length * this.config.lineHeight;
    pageHeight =
        this.config.marginTop +
        contentHeight +

```

```

        this.config.marginBottom;

        page = this.createPageWithLayout(pdfDoc, pageHeight);
        currentY = pageHeight - this.config.marginTop;
    } else {
        page = this.createPageWithLayout(pdfDoc);
        pageHeight = page.getSize().height;
        currentY = pageHeight - this.config.marginTop;
    }

    /**
     * RENDER TO CANVAS SURFACE
     */
    for (const wrappedLine of allFinalWrappedLines) {
        if (
            !this.config.oneLongPage &&
            currentY - this.config.fontSize < pageBottomThreshold
        ) {
            page = this.createPageWithLayout(pdfDoc);
            currentY = page.getSize().height - this.config.marginTop;
        }

        let currentX: number = this.config.marginLeft;

        for (const chunk of wrappedLine.atoms) {
            if (chunk.text.length > 0) {
                page.drawText(chunk.text, {
                    x: currentX,
                    y: currentY,
                    size: this.config.fontSize,
                    font: font,
                    color: chunk.color,
                    opacity: this.config.textOpacity,
                });
                currentX += font.widthOfTextAtSize(
                    chunk.text,
                    this.config.fontSize,
                );
            }
        }

        currentY -= this.config.lineHeight;
    }

    const pdfBytes: Uint8Array = await pdfDoc.save();
    await Bun.write(outputPath, pdfBytes);
}

private wrapText(
    text: string,
    maxWidth: number,
    getTextWidth: TextWidth,
): string[] {
    if (this.config.wordWrap === 'none') return [text];
    const normalizedText: string = text.replace(/\t/g, ' ');

    // CHARACTER WRAPPING
    if (this.config.wordWrap === 'char') {
        const lines: string[] = [];
        let currentLine: string = '';
        for (const char of normalizedText) {
            if (getTextWidth(currentLine + char) > maxWidth) {
                lines.push(currentLine);
            }
        }
    }
}

```

```

        currentLine = char;
    } else {
        currentLine += char;
    }
}
if (currentLine) lines.push(currentLine);
return lines.length > 0 ? lines : [''];
}

// WORD WRAPPING
const leadingSpaceMatch: RegExpMatchArray | null =
    normalizedText.match(/^ +/);
const leadingSpaces: string = leadingSpaceMatch
    ? leadingSpaceMatch[0]
    : '';
const remainingText: string = normalizedText.slice(
    leadingSpaces.length,
);

// Split by spaces but preserve empty strings to maintain multiple spaces
const words: string[] = remainingText.split(' ');
const lines: string[] = [];
let currentLine: string = leadingSpaces;

for (const word of words) {
    // Determine what the line would look like if we add this word
    const isLineEmpty: boolean =
        currentLine === leadingSpaces || currentLine === '';
    const testLine: string = isLineEmpty
        ? `${currentLine}${word}`
        : `${currentLine} ${word}`;

    if (getTextWidth(testLine) <= maxWidth) {
        currentLine = testLine;
    } else {
        // The word doesn't fit on the current line.
        // If we've already added words to this line, push it and move the word to the next line.
        if (!isLineEmpty) {
            lines.push(currentLine);
            // Start next line with leading spaces preserved
            currentLine = `${leadingSpaces}${word}`;

            // Check if the single word itself is wider than the layout page width
            if (getTextWidth(currentLine) > maxWidth) {
                // Fallback: forced break of this massive single word
                lines.push(currentLine);
                currentLine = leadingSpaces;
            }
        } else {
            // If the single word is already at the start of a line and STILL exceeds maxWidth,
            // we have no choice but to force-push it so it doesn't cause an infinite loop.
            lines.push(testLine);
            currentLine = leadingSpaces;
        }
    }
}

if (currentLine && currentLine !== leadingSpaces) {
    lines.push(currentLine);
}

return lines.length > 0 ? lines : [''];
}

```

```

private async getFontSafely(
    fontPath: string,
    pdfDoc: PDFDocument,
): Promise<PDFFont> {
    try {
        const absoluteFontPath: string = path.resolve(
            import.meta.dir,
            fontPath,
        );
        const fontFile: BunFile = Bun.file(absoluteFontPath);
        if (!(await fontFile.exists())) {
            throw new Error(
                `File not found at target resolution layout: "${absoluteFontPath}"`,
            );
        }
        const fontBytes: ArrayBuffer = await fontFile.arrayBuffer();
        return pdfDoc.embedFont(fontBytes); // No 'await' needed here when returning directly
    } catch (e: unknown) {
        if (e instanceof Error) {
            console.warn(
                `[ToPdf Warning]: Fallback to standard Monospace Courier used.${e.message}`,
            );
        } else {
            // Optional: Catches edge cases where something that isn't an Error object was thrown
            console.warn(
                `[ToPdf Warning]: Fallback to standard Monospace Courier used. Unknown error occurred`,
            );
        }
        return pdfDoc.embedFont('Courier'); // No 'await' needed here either
    }
}

private async getText(sourcePath: string): Promise<string> {
    const sourceFile: BunFile = Bun.file(sourcePath);
    if (!(await sourceFile.exists())) {
        throw new Error(
            `Input text source file completely missing at path: "${sourcePath}"`,
        );
    }
    return sourceFile.text();
}

export class ToPdf {
    public static create(options?: Config): ToPdfService {
        const mergedConfig: Required<Config> = {
            ...DEFAULT_CONFIG,
            ...options,
        };
        return new ToPdfService(mergedConfig);
    }
}

// ██████████< END >-██████████
// FILE NAME: : ToPdf.ts
// TYPE      : .TS
// PATH      : utils/ToPdf.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >-██████████

// ██████████< START >-██████████
// FILE NAME: : censor.ts

```

```

// TYPE      : .TS
// PATH      : utils/censor.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< START >────────────────────────────────────────██████████

import { join } from 'path';

// Get command line arguments
const [, , filePath] = Bun.argv;

if (!filePath) {
  console.error(
    '✖Please provide a file path. Usage: bun censor.ts <file-path>',
  );
  process.exit(1);
}

try {
  // Resolve absolute path and load file
  const absolutePath = join(process.cwd(), filePath);
  const file = Bun.file(absolutePath);

  if (!(await file.exists())) {
    console.error(`✖File not found: ${filePath}`);
    process.exit(1);
  }

  const text = await file.text();

  // Regex to match URLs (http, https, and www)
  const urlRegex = /https?:\/\/[^\s$.?#].[^\s]*|www\.[^\s$.?#].[^\s]*/gi;

  // Replace links and track changes
  let count = 0;
  const censoredText = text.replace(urlRegex, () => {
    count++;
    return 'WEBSITE';
  });

  // Write changes back to the same file
  await Bun.write(absolutePath, censoredText);

  console.log(
    `✅Success! Replaced ${count} website address(es) in ${filePath}`,
  );
} catch (error) {
  console.error(
    '✖An error occurred:',
    error instanceof Error ? error.message : error,
  );
}

// ██████████< END >────────────────────────────────────────██████████
// FILE NAME: : censor.ts
// TYPE      : .TS
// PATH      : utils/censor.ts
// PROCESSED: 2026-09-06T21:55:30.013Z
// ██████████< END >────────────────────────────────────────██████████

```